

t_{10} have the common subexpressions $4*i$ and $4*j$, respectively, on the right side in Fig. 10.6(a). They have been eliminated in Fig. 10.6(b), by using t_6 instead of t_7 and t_8 instead of t_{10} . This change is what would result if we reconstructed the intermediate code from the dag for the basic block.

Example 10.2. Figure 10.7 shows the result of eliminating both global and local common subexpressions from blocks B_5 and B_6 in the flow graph of Fig. 10.5. We first discuss the transformation of B_5 and then mention some subtleties involving arrays.

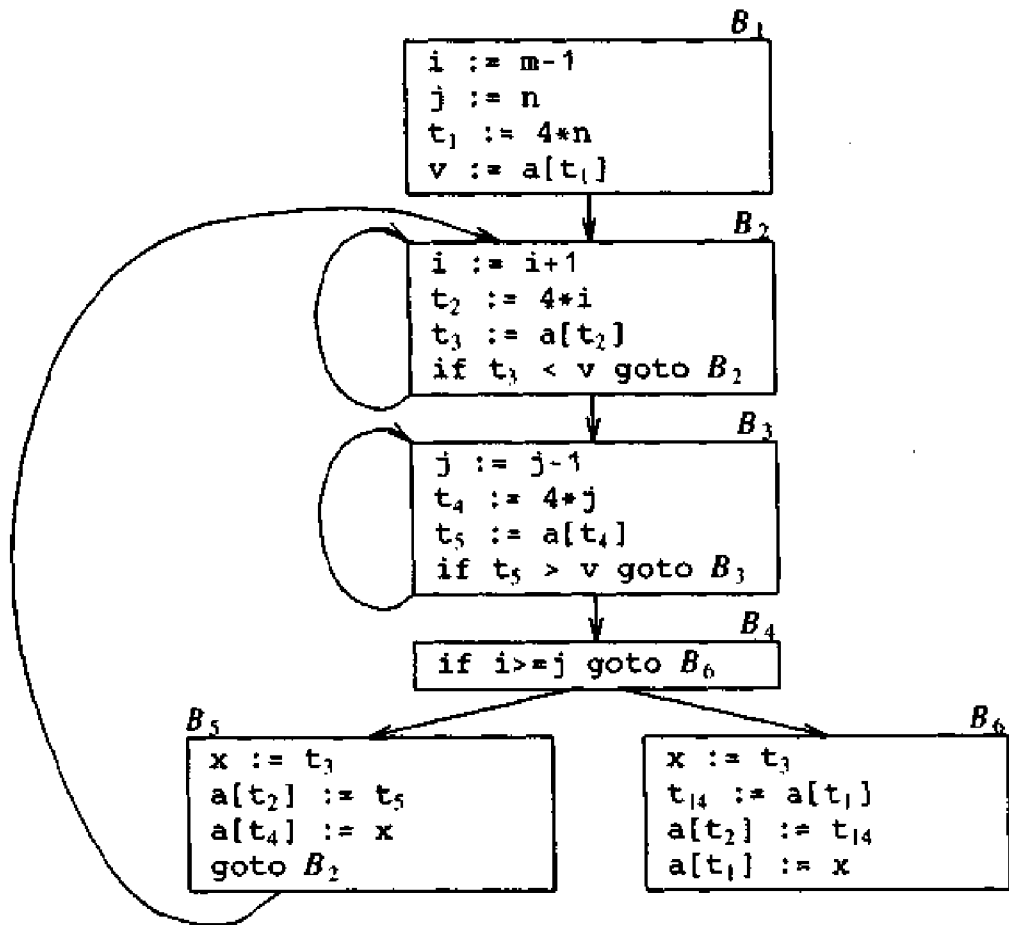


Fig. 10.7. B_5 and B_6 after common subexpression elimination.

After local common subexpressions are eliminated, B_5 still evaluates $4*i$ and $4*j$, as shown in Fig. 10.6(b). Both are common subexpressions; in particular, the three statements

$$t_8 := 4*j; \quad t_9 := a[t_8]; \quad a[t_8] := x$$

in B_5 can be replaced by

$$t_9 := a[t_4]; \quad a[t_4] := x$$

using t_4 computed in block B_3 . In Fig. 10.7, observe that as control passes from the evaluation of $4*j$ in B_3 to B_5 , there is no change in j , so t_4 can be used if $4*j$ is needed.

Another common subexpression comes to light in B_5 after t_4 replaces t_8 . The new expression $a[t_4]$ corresponds to the value of $a[j]$ at the source level. Not only does j retain its value as control leaves B_3 and then enters B_5 , but $a[j]$, a value computed into a temporary t_5 , does too because there are no assignments to elements of the array a in the interim. The statements

$$t_9 := a[t_4]; \quad a[t_6] := t_9$$

in B_5 can therefore be replaced by

$$a[t_6] := t_5$$

Analogously, the value assigned to x in block B_5 of Fig. 10.6(b) is seen to be the same as the value assigned to t_3 in block B_2 . Block B_5 in Fig. 10.7 is the result of eliminating common subexpressions corresponding to the values of the source level expressions $a[i]$ and $a[j]$ from B_5 in Fig. 10.6(b). A similar series of transformations has been done to B_6 in Fig. 10.7.

The expression $a[t_1]$ in blocks B_1 and B_6 of Fig. 10.7 is not considered a common subexpression, although t_1 can be used in both places. After control leaves B_1 and before it reaches B_6 , it can go through B_5 , where there are assignments to a . Hence, $a[t_1]$ may not have the same value on reaching B_6 as it did on leaving B_1 , and it is not safe to treat $a[t_1]$ as a common subexpression. $\square$

Copy Propagation

Block B_5 in Fig. 10.7 can be further improved by eliminating x using two new transformations. One concerns assignments of the form $f:=g$ called *copy statements*, or *copies* for short. Had we gone into more detail in Example 10.2, copies would have arisen much sooner, because the algorithm for eliminating common subexpressions introduces them, as do several other algorithms. For example, when the common subexpression in $c:=d+e$ is eliminated in Fig. 10.8, the algorithm uses a new variable t to hold the value of $d+e$. Since control may reach $c:=d+e$ either after the assignment to a or after the assignment to b , it would be incorrect to replace $c:=d+e$ by either $c:=a$ or by $c:=b$.

The idea behind the copy-propagation transformation is to use g for f , wherever possible after the copy statement $f:=g$. For example, the assignment $x:=t_3$ in block B_5 of Fig. 10.7 is a copy. Copy propagation applied to B_5 yields:

$$\begin{aligned} x &:= t_3 \\ a[t_2] &:= t_5 \\ a[t_4] &:= t_3 \\ \text{goto } B_2 \end{aligned} \tag{10.1}$$

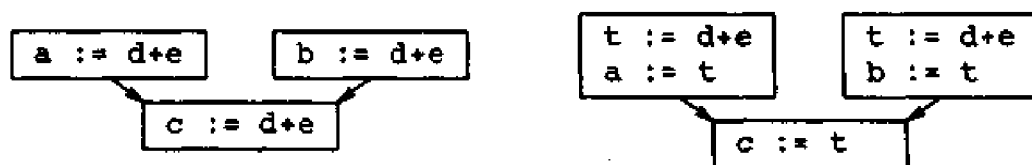


Fig. 10.8. Copies introduced during common subexpression elimination.

This may not appear to be an improvement, but as we shall see, it gives us the opportunity to eliminate the assignment to x .

Dead-Code Elimination

A variable is live at a point in a program if its value can be used subsequently; otherwise, it is dead at that point. A related idea is dead or useless code, statements that compute values that never get used. While the programmer is unlikely to introduce any dead code intentionally, it may appear as the result of previous transformations. For example, in Section 9.9 we discussed the use of `debug` that is set to true or false at various points in the program, and used in statements like

```
if (debug) print ... (10.2)
```

By a data-flow analysis, it may be possible to deduce that each time the program reaches this statement, the value of `debug` is false. Usually, it is because there is one particular statement

```
debug := false
```

that we can deduce to be the last assignment to `debug` prior to the test (10.2), no matter what sequence of branches the program actually takes. If copy propagation replaces `debug` by false, then the print statement is dead because it cannot be reached. We can eliminate both the test and printing from the object code. More generally, deducing at compile time that the value of an expression is a constant and using the constant instead is known as *constant folding*.

One advantage of copy propagation is that it often turns the copy statement into dead code. For example, copy propagation followed by dead-code elimination removes the assignment to x and transforms (10.1) into:

```
a[t2] := t5
a[t4] := t3
goto B2
```

This code is a further improvement of block B_5 in Fig. 10.7.

Loop Optimizations

We now give a brief introduction to a very important place for optimizations, namely loops, especially the inner loops where programs tend to spend the bulk of their time. The running time of a program may be improved if we decrease the number of instructions in an inner loop, even if we increase the amount of code outside that loop. Three techniques are important for loop optimization: *code motion*, which moves code outside a loop; *induction-variable elimination*, which we apply to eliminate i and j from the inner loops B_2 and B_3 of Fig. 10.7; and, *reduction in strength*, which replaces an expensive operation by a cheaper one, such as a multiplication by an addition.

Code Motion

An important modification that decreases the amount of code in a loop is code motion. This transformation takes an expression that yields the same result independent of the number of times a loop is executed (a *loop-invariant computation*) and places the expression before the loop. Note that the notion "before the loop" assumes the existence of an entry for the loop. For example, evaluation of $\text{limit}-2$ is a loop-invariant computation in the following while-statement:

```
while ( i <= limit-2 ) /* statement does not change limit */
```

Code motion will result in the equivalent of

```
t = limit-2;
while ( i <= t ) /* statement does not change limit or t */
```

Induction Variables and Reduction in Strength

While code motion is not applicable to the quicksort example we have been considering, the other two transformations are. Loops are usually processed inside out. For example, consider the loop around B_3 . Only the portion of the flow graph relevant to the transformations on B_3 is shown in Fig. 10.9.

Note that the values of j and t_4 remain in lock-step; every time the value of j decreases by 1, that of t_4 decreases by 4 because $4*j$ is assigned to t_4 . Such identifiers are called *induction variables*.

When there are two or more induction variables in a loop, it may be possible to get rid of all but one, by the process of induction-variable elimination. For the inner loop around B_3 in Fig. 10.9(a), we cannot get rid of either j or t_4 completely; t_4 is used in B_3 and j in B_4 . However, we can illustrate reduction in strength and illustrate a part of the process of induction-variable elimination. Eventually, j will be eliminated when the outer loop of B_2-B_5 is considered.

Example 10.3. As the relationship $t_4 = 4*j$ surely holds after such an assignment to t_4 in Fig. 10.9(a), and t_4 is not changed elsewhere in the inner loop around B_3 , it follows that just after the statement $j:=j-1$ the

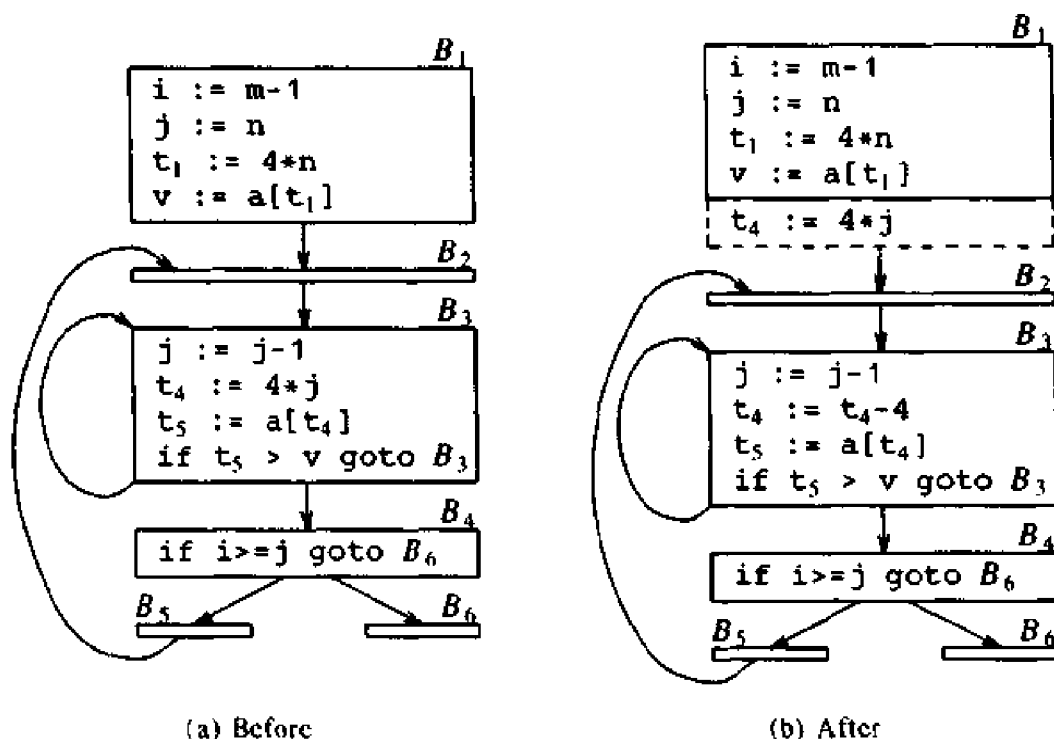


Fig. 10.9. Strength reduction applied to $4*j$ in block B_3 .

relationship $t_4 = 4*j - 4$ must hold. We may therefore replace the assignment $t_4 := 4*j$ by $t_4 := t_4 - 4$. The only problem is that t_4 does not have a value when we enter block B_3 for the first time. Since we must maintain the relationship $t_4 = 4*j$ on entry to the block B_3 , we place an initialization of t_4 at the end of the block where j itself is initialized, shown by the dashed addition to block B_1 in Fig. 10.9(b).

The replacement of a multiplication by a subtraction will speed up the object code if multiplication takes more time than addition or subtraction, as is the case on many machines. $\square$

Section 10.7 discusses how induction variables can be detected and what transformations can be applied. We conclude this section with one more example of induction-variable elimination that treats i and j in the context of the outer loop containing B_2 , B_3 , B_4 , and B_5 .

Example 10.4. After reduction in strength is applied to the inner loops around B_2 and B_3 , the only use of i and j is to determine the outcome of the test in block B_4 . We know that the values of i and t_2 satisfy the relationship $t_2 = 4*i$, while those of j and t_4 satisfy the relationship $t_4 = 4*j$, so the test $t_2 \geq t_4$ is equivalent to $i \geq j$. Once this replacement is made, i in block B_2 and j in block B_3 become dead variables and the assignments to them in these blocks become dead code that can be eliminated, resulting in the flow graph shown in Fig. 10.10. $\square$

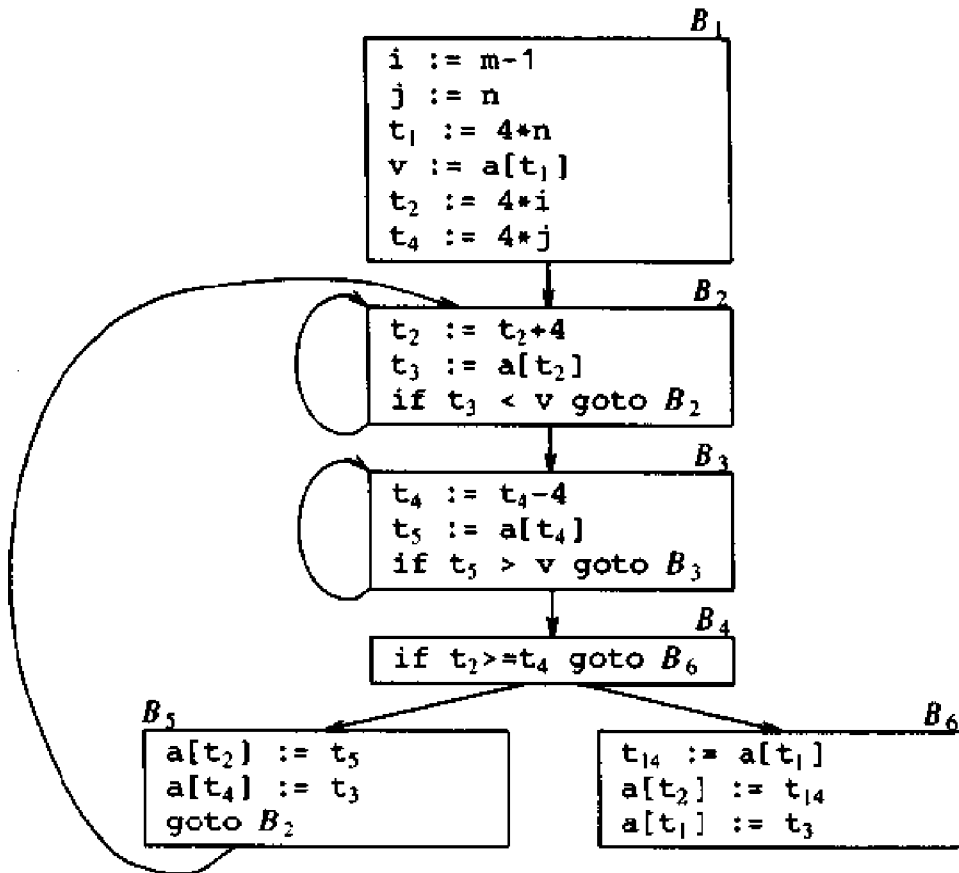


Fig. 10.10. Flow graph after induction-variable elimination.

The code-improving transformations have been effective. In Fig. 10.10, the number of instructions in blocks B_2 and B_3 has been reduced from 4 to 3 from the original flow graph in Fig. 10.5, in B_5 it has been reduced from 9 to 3, and in B_6 from 8 to 3. True, B_1 has grown from four instructions to six, but B_1 is executed only once in the fragment, so the total running time is barely affected by the size of B_1 .

10.3 OPTIMIZATION OF BASIC BLOCKS

In Chapter 9, we saw a number of code-improving transformations for basic blocks. These included structure-preserving transformations, such as common subexpression elimination and dead-code elimination, and algebraic transformations such as reduction in strength.

Many of the structure-preserving transformations can be implemented by constructing a dag for a basic block. Recall that there is a node in the dag for each of the initial values of the variables appearing in the basic block, and there is a node n associated with each statement s within the block. The children of n are those nodes corresponding to statements that are the last definitions prior to s of the operands used by s . Node n is labeled by the operator

applied at s , and also attached to n is the list of variables for which it is the last definition within the block. We also note those nodes, if any, whose values are live on exit from the block; these are the output nodes.

Common subexpressions can be detected by noticing, as a new node m is about to be added, whether there is an existing node n with the same children, in the same order, and with the same operator. If so, n computes the same value as m and may be used in its place.

Example 10.5. A dag for the block (10.3)

$a := b + c$
 $b := a - d$
 $c := b + c$
 $d := a - d$
(10.3)

is shown in Fig. 10.11. When we construct the node for the third statement $c := b + c$, we know that the use of b in $b + c$ refers to the node of Fig. 10.11 labeled $-$, because that is the most recent definition of b . Thus, we do not confuse the values computed at statements one and three.

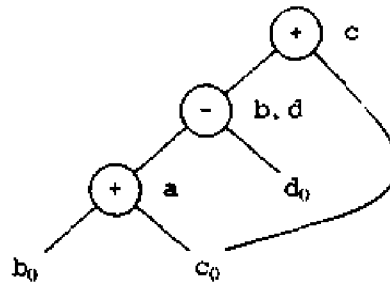


Fig. 10.11. Dag for basic block (10.3).

However, the node corresponding to the fourth statement $d := a - d$ has the operator $-$ and the nodes labeled a and d_0 as children. Since the operator and the children are the same as those for the node corresponding to statement two, we do not create this node, but add d to the list of definitions for the node labeled $-$. □

It might appear that, since there are only three nodes in the dag of Fig. 10.11, block (10.3) can be replaced by a block with only three statements. In fact, if either b or d is not live on exit from the block, then we do not need to compute that variable, and can use the other to receive the value represented by the node labeled $-$ in Fig. 10.11. For example, if b is not live on exit, we could use:

$a := b + c$
 $d := a - d$
 $c := d + c$

However, if both b and d are live on exit, then a fourth statement must be used to copy the value from one to the other.²

Note that when we look for common subexpressions, we really are looking for expressions that are guaranteed to compute the same value, no matter how that value is computed. Thus, the dag method will miss the fact that the expression computed by the first and fourth statements in the sequence

$$\begin{array}{l} a := b + c \\ b := b - d \\ c := c + d \\ e := b + c \end{array} \quad (10.4)$$

is the same, namely, $b+c$. However, algebraic identities applied to the dag, as discussed next, may expose the equivalence. The dag for this sequence is shown in Fig. 10.12.

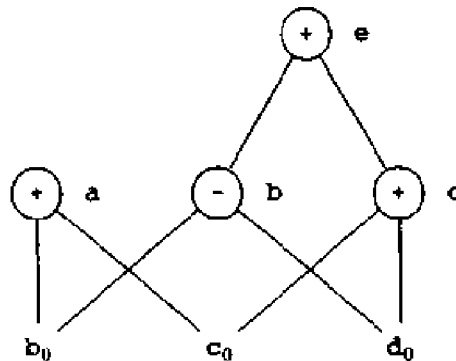


Fig. 10.12. Dag for basic block (10.4).

The operation on dags that corresponds to dead-code elimination is quite straightforward to implement. We delete from a dag any root (node with no ancestors) that has no live variables. Repeated application of this transformation will remove all nodes from the dag that correspond to dead code.

The Use of Algebraic Identities

Algebraic identities represent another important class of optimizations on basic blocks. In Section 9.9, we introduced some simple algebraic transformations that one might try during optimization. For example, we may apply arithmetic identities, such as

² In general, we have to be careful when reconstructing code from dags to choose the names of variables corresponding to nodes carefully. If a variable x is defined twice, or if it is assigned once and the initial value x_0 is also used, then we must make sure that we do not change the value of x until we have made all uses of the node whose value x previously held.

```

x + 0 = 0 + x = x
x - 0 = x
x * 1 = 1 * x = x
x / 1 = x

```

Another class of algebraic optimizations includes reduction in strength, that is, replacing a more expensive operator by a cheaper one as in

```

x ** 2 = x * x
2.0 * x = x + x
x / 2 = x * 0.5

```

A third class of related optimizations is constant folding. Here we evaluate constant expressions at compile time and replace the constant expressions by their values.³ Thus the expression $2*3.14$ would be replaced by 6.28 . Many constant expressions arise through the use of symbolic constants.

The dag-construction process can help us apply these and other more general algebraic transformations such as commutativity and associativity. For example, suppose $*$ is commutative; that is, $x*y = y*x$. Before we create a new node labeled $*$ with left child m and right child n , we check whether such a node already exists. We then check for a node having operator $*$, left child n and right child m .

The relational operators $\leq$, $\geq$, $<$, $>$, $=$, and $\neq$ sometimes generate unexpected common subexpressions. For example, the condition $x > y$ can also be tested by subtracting the arguments and performing a test on the condition code set by the subtraction. (The subtraction can, however, introduce overflows and underflows while a compare instruction would not.) Thus, only one node of the dag need be generated for $x - y$ and $x > y$.

Associative laws may also be applied to expose common subexpressions. For example, if the source code has the assignments

```

a := b + c
e := c + d + b

```

the following intermediate code might be generated:

```

a := b + c
t := c + d
e := t + b

```

If t is not needed outside this block, we can change this sequence to

```

a := b + c
e := a + d

```

³ Arithmetic expressions should be evaluated the same way at compile time as they are at run time. K. Thompson has suggested an elegant solution to constant folding: compile the constant expression, execute the target code on the spot, and replace the expression with the result. Thus, the compiler does not need to contain an interpreter.

using both the associativity and commutativity of $+$.

The compiler writer should examine the language specification carefully to determine what rearrangements of computations are permitted, since computer arithmetic does not always obey the algebraic identities of mathematics. For example, the standard for Fortran 77 states that a compiler may evaluate any mathematically equivalent expression, provided that the integrity of parentheses is not violated. Thus, a compiler may evaluate $x*y-x*z$ as $x*(y-z)$ but it may not evaluate $a+(b-c)$ as $(a+b)-c$. A Fortran compiler must therefore keep track of where parentheses were present in the source language expressions, if it is to optimize programs in accordance with the language definition.

10.4 LOOPS IN FLOW GRAPHS

Before considering loop optimizations, we need to define what constitutes a loop in a flow graph. We shall use the notion of a node "dominating" another to define "natural loops" and the important special class of "reducible" flow graphs. An algorithm for finding dominators and checking reducibility of flow graphs will be given in Section 10.9.

Dominators

We say node d of a flow graph *dominates* node n , written $d \text{ dom } n$, if every path from the initial node of the flow graph to n goes through d . Under this definition, every node dominates itself, and the entry of a loop (as defined in Section 9.4) dominates all nodes in the loop.

Example 10.6. Consider the flow graph of Fig. 10.13, with initial node 1. The initial node dominates every node. Node 2 dominates only itself, since control can reach any other node along a path that begins $1 \rightarrow 3$. Node 3 dominates all but 1 and 2. Node 4 dominates all but 1, 2 and 3, since all paths from 1 must begin $1 \rightarrow 2 \rightarrow 3 \rightarrow 4$ or $1 \rightarrow 3 \rightarrow 4$. Nodes 5 and 6 dominate only themselves, since flow of control can skip around either by going through the other. Finally, 7 dominates 7, 8, 9, 10; 8 dominates 8, 9, 10; 9 and 10 dominate only themselves. $\square$

A useful way of presenting dominator information is in a tree, called the *dominator tree*, in which the initial node is the root, and each node d dominates only its descendants in the tree. For example, Fig. 10.14 shows the dominator tree for the flow graph of Fig. 10.13.

The existence of dominator trees follows from a property of dominators; each node n has a unique *immediate dominator* m that is the last dominator of n on any path from the initial node to n . In terms of the *dom* relation, the immediate dominator m has that property that if $d \neq n$ and $d \text{ dom } n$, then $d \text{ dom } m$.

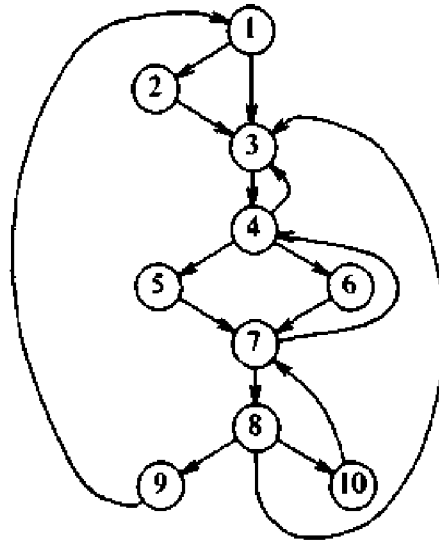


Fig. 10.13. Flow graph.

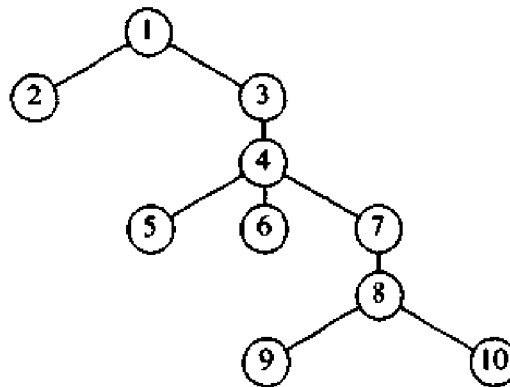


Fig. 10.14. Dominator tree for flow graph of Fig. 10.13.

Natural Loops

One important application of dominator information is in determining the loops of a flow graph suitable for improvement. There are two essential properties of such loops.

1. A loop must have a single entry point, called the "header." This entry point dominates all nodes in the loop, or it would not be the sole entry to the loop.
2. There must be at least one way to iterate the loop, i.e., at least one path back to the header.

A good way to find all the loops in a flow graph is to search for edges in

the flow graph whose heads dominate their tails. (If $a \rightarrow b$ is an edge, b is the *head* and a is the *tail*.) We call such edges *back edges*.

Example 10.7. In Fig. 10.13, there is an edge $7 \rightarrow 4$, and $4 \text{ dom } 7$. Similarly, $10 \rightarrow 7$ is an edge, and $7 \text{ dom } 10$. The other edges with this property are $4 \rightarrow 3$, $8 \rightarrow 3$, and $9 \rightarrow 1$. Note that these are exactly the edges that one would think of as forming loops in the flow graph. $\square$

Given a back edge $n \rightarrow d$, we define the *natural loop* of the edge to be d plus the set of nodes that can reach n without going through d . Node d is the header of the loop.

Example 10.8. The natural loop of the edge $10 \rightarrow 7$ consists of nodes 7, 8, and 10, since 8 and 10 are all those nodes that can reach 10 without going through 7. The natural loop of $9 \rightarrow 1$ is the entire flow graph. (Don't forget the path $10 \rightarrow 7 \rightarrow 8 \rightarrow 9$!) $\square$

Algorithm 10.1. Constructing the natural loop of a back edge.

Input. A flow graph G and a back edge $n \rightarrow d$.

Output. The set *loop* consisting of all nodes in the natural loop of $n \rightarrow d$.

Method. Beginning with node n , we consider each node $m \neq d$ that we know is in *loop*, to make sure that m 's predecessors are also placed in *loop*. The algorithm is given in Fig. 10.15. Each node in *loop*, except for d , is placed once on *stack*, so its predecessors will be examined. Note that because d is put in the loop initially, we never examine its predecessors, and thus find only those nodes that reach n without going through d . $\square$

```

procedure insert( $m$ );
if  $m$  is not in loop then begin
     $loop := loop \cup \{m\}$ ;
    push  $m$  onto stack
end;

/* main program follows */

 $stack := \text{empty}$ ;
 $loop := \{d\}$ ;
insert( $n$ );
while stack is not empty do begin
    pop  $m$ , the first element of stack, off stack;
    for each predecessor  $p$  of  $m$  do insert( $p$ )
end

```

Fig. 10.15. Algorithm for constructing the natural loop.

Inner Loops

If we use the natural loops as "the loops," then we have the useful property that unless two loops have the same header, they are either disjoint or one is entirely contained (*nested within*) the other. Thus, neglecting loops with the same header for the moment, we have a natural notion of *inner loop*: one that contains no other loops.

When two loops have the same header, as in Fig. 10.16, it is hard to tell which is the inner loop. For example, if the test at the end of B_1 were

if $a = 10$ goto B_2

probably the loop $\{B_0, B_1, B_3\}$ would be the inner loop. However, we could not be sure without a detailed examination of the code. Perhaps a is almost always 10, and it is typical to go around the $\{B_0, B_1, B_2\}$ loop many times before branching to B_3 . Thus, we shall assume that when two natural loops have the same header, but neither is nested within the other, they are combined and treated as a single loop.

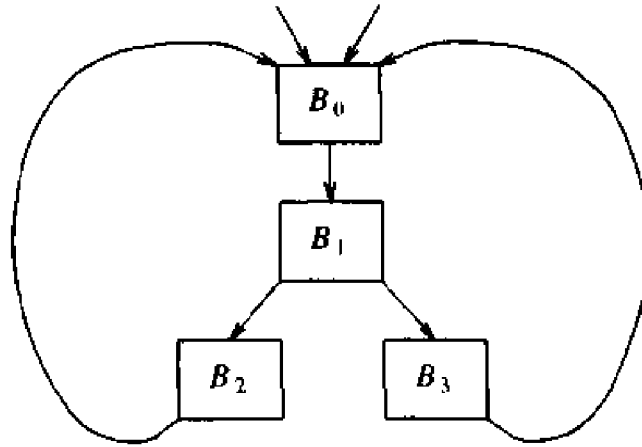


Fig. 10.16. Two loops with the same header.

Pre-Headers

Several transformations require us to move statements "before the header." We therefore begin treatment of a loop L by creating a new block, called the *preheader*. The preheader has only the header as successor, and all edges which formerly entered the header of L from outside L instead enter the preheader. Edges from inside loop L to the header are not changed. The arrangement is shown in Fig. 10.17. Initially, the preheader is empty, but transformations on L may place statements in it.

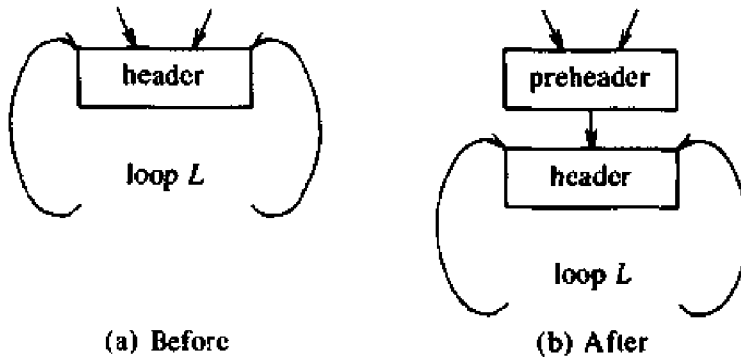


Fig. 10.17. Introduction of the preheader.

Reducible Flow Graphs

Flow graphs that occur in practice frequently fall into the class of reducible flow graphs defined below. Exclusive use of structured flow-of-control statements such as if-then-else, while-do, continue, and break statements produces programs whose flow graphs are always reducible. Even programs written using goto statements by programmers with no prior knowledge of structured program design are almost always reducible.

A variety of definitions of “reducible flow graph” have been proposed. The one we adopt here brings out one of the most important properties of reducible flow graphs; namely, that there are no jumps into the middle of loops from outside; the only entry to a loop is through its header. The exercises and bibliographic notes contain a brief history of the concept.

A flow graph G is *reducible* if and only if we can partition the edges into two disjoint groups, often called the *forward* edges and *back* edges, with the following two properties:

1. The forward edges form an acyclic graph in which every node can be reached from the initial node of G .
2. The back edges consist only of edges whose heads dominate their tails.

Example 10.9. The flow graph of Fig. 10.13 is reducible. In general, if we know the relation *dom* for a flow graph, we can find and remove all the back edges. The remaining edges must be the forward edges if the graph is reducible, and to check whether a flow graph is reducible it suffices to check that the forward edges form an acyclic graph. In the case of Fig. 10.13, it is easy to check that if we remove the five back edges $4 \rightarrow 3$, $7 \rightarrow 4$, $8 \rightarrow 3$, $9 \rightarrow 1$, and $10 \rightarrow 7$, whose heads dominate their tails, the remaining graph is acyclic. $\square$

Example 10.10. Consider the flow graph of Fig. 10.18, whose initial node is 1. This flow graph has no back edges, since no head of an edge dominates the tail of that edge. Thus it could only be reducible if the entire graph were acyclic. But since it is not, the flow graph is not reducible. Intuitively, the reason this flow graph is not reducible is that the cycle 2-3 can be entered at

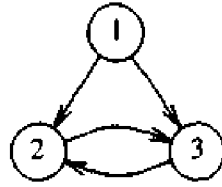


Fig. 10.18. A nonreducible flow graph.

two different places, nodes 2 and 3. □

The key property of reducible flow graphs for loop analysis is that in such flow graphs every set of nodes that we would informally regard as a loop must contain a back edge. In fact, we need examine only the natural loops of back edges in order to find all loops in a program whose flow graph is reducible. In contrast, the flow graph of Fig. 10.18 appears to have a “loop” consisting of nodes 2 and 3, but there is no back edge of which this is the natural loop. In fact, that “loop” has two headers, 2 and 3, making application of many code-optimization techniques, such as those introduced in Section 10.2 for code motion and induction variable removal, not directly applicable.

Fortunately, nonreducible control-flow structures, such as that in Fig. 10.18, appear so rarely in most programs as to make the study of loops with more than one header of secondary importance. There are even languages, such as Bliss and Modula 2, that allow only programs with reducible flow graphs, and many other languages will yield only reducible flow graphs as long as we use no goto's.

Example 10.11. Returning again to Fig. 10.13, we note that the only “inner loop,” that is, a loop with no subloops, is $\{7, 8, 10\}$, the natural loop of back edge $10 \rightarrow 7$. The set $\{4, 5, 6, 7, 8, 10\}$ is the natural loop of $7 \rightarrow 4$. (Note that 8 and 10 can reach 7 via edge $10 \rightarrow 7$.) Our intuition that $\{4, 5, 6, 7\}$ forms a loop is wrong, since 4 and 7 would both be entries from outside, violating our single-entry requirement. Put another way, there is no reason to assume that control spends much time going around the set of nodes $\{4, 5, 6, 7\}$; it is just as plausible that control passes to 8 from 7 more often than it does to 4. By including 8 and 10 in the loop, we are more certain of having isolated a heavily traveled region of the program.

It is wise to recognize, however, the danger in making assumptions about the frequency of branches. For example, if we moved an invariant statement out of 8 or 10 in loop $\{7, 8, 10\}$, and in fact, control followed edge $7 \rightarrow 4$ more frequently than $7 \rightarrow 8$, we would actually increase the number of times the moved statement was executed. We shall discuss methods to avoid this problem in Section 10.7.

The next larger loop is $\{3, 4, 5, 6, 7, 8, 10\}$, which is the natural loop of both edges $4 \rightarrow 3$ and $8 \rightarrow 3$. As before, our intuition that $\{3, 4\}$ should be regarded as a loop violates the single header requirement. The last loop, the one for back edge $9 \rightarrow 1$, is the entire flow graph. □

There are several additional useful properties of reducible flow graphs, which we shall introduce when we discuss the topics of depth-first search and interval analysis in Section 10.9.

10.5 INTRODUCTION TO GLOBAL DATA-FLOW ANALYSIS

In order to do code optimization and a good job of code generation, a compiler needs to collect information about the program as a whole and to distribute this information to each block in the flow graph. For example, we saw in Section 9.7 how knowing what variables are live on exit from each block could improve register usage. Section 10.2 suggested how we could use knowledge of global common subexpressions to eliminate redundant computations. Likewise, Sections 9.9 and 10.3 discussed how a compiler could take advantage of "reaching definitions," such as knowing where a variable like `debug` was last defined before reaching a given block, in order to perform transformations like constant folding and dead-code elimination. These facts are just a few examples of *data-flow information* that an optimizing compiler collects by a process known as *data-flow analysis*.

Data-flow information can be collected by setting up and solving systems of equations that relate information at various points in a program. A typical equation has the form

$$out[S] = gen[S] \cup (in[S] - kill[S]) \quad (10.5)$$

and can be read as, "the information at the end of a statement is either generated within the statement, or enters at the beginning and is not killed as control flows through the statement." Such equations are called *data-flow equations*.

The details of how data-flow equations are set up and solved depend on three factors.

1. The notions of generating and killing depend on the desired information, i.e., on the data-flow analysis problem to be solved. Moreover, for some problems, instead of proceeding along with the flow of control and defining $out[S]$ in terms of $in[S]$, we need to proceed backwards and define $in[S]$ in terms of $out[S]$.
2. Since data flows along control paths, data-flow analysis is affected by the control constructs in a program. In fact, when we write $out[S]$ we implicitly assume that there is unique end point where control leaves the statement; in general, equations are set up at the level of basic blocks rather than statements, because blocks do have unique end points.
3. There are subtleties that go along with such statements as procedure calls, assignments through pointer variables, and even assignments to array variables.

In this section, we consider the problem of determining the set of

definitions reaching a point in a program and its use in finding opportunities for constant folding. Later in this chapter, algorithms for code motion and induction variable elimination will also use this information.

We initially consider programs constructed using **if** and **do-while** statements. The predictable control flow in these statements allows us to concentrate on the ideas needed to set up and solve data-flow equations. Assignments in this section are either copy statements or are of the form $a := b + c$. In this chapter, we frequently use “+” as a typical operator. All that we say applies straightforwardly to other operators, including those with one operand, or with more than two operands.

Points and Paths

Within a basic block, we talk of the *point* between two adjacent statements, as well as the point before the first statement and after the last. Thus, block B_1 in Fig. 10.19 has four points: one before any of the assignments and one after each of the three assignments.

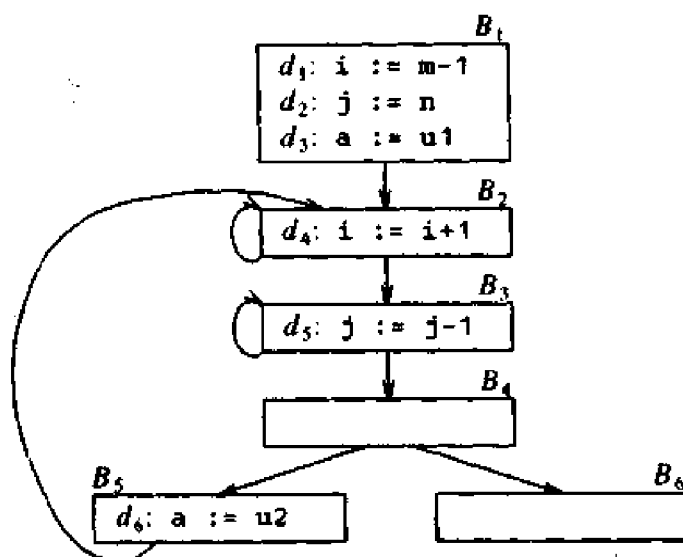


Fig. 10.19. A flow graph.

Now, let us take a global view and consider all the points in all the blocks. A *path* from p_1 to p_n is a sequence of points $p_1, p_2, \dots, p_n$ such that for each i between 1 and $n-1$, either

1. p_i is the point immediately preceding a statement and p_{i+1} is the point immediately following that statement in the same block, or
2. p_i is the end of some block and p_{i+1} is the beginning of a successor block.

Example 10.12. In Fig. 10.19 there is a path from the beginning of block B_5 to the beginning of block B_6 . It travels through the end point of B_5 and then

through all the points in B_2 , B_3 , and B_4 , in order, before it reaches the beginning of B_6 . $\square$

Reaching Definitions

A *definition* of a variable x is a statement that assigns, or may assign, a value to x . The most common forms of definition are assignments to x and statements that read a value from an I/O device and store it in x . These statements certainly define a value for x , and they are referred to as *unambiguous* definitions of x . There are certain other kinds of statements that may define a value for x ; they are called *ambiguous* definitions. The most usual forms of ambiguous definitions of x are:

1. A call of a procedure with x as a parameter (other than a by-value parameter) or a procedure that can access x because x is in the scope of the procedure. We also have to consider the possibility of "aliasing," where x is not in the scope of the procedure, but x has been identified with another variable that is passed as a parameter or is in the scope. These issues are taken up in Section 10.8.
2. An assignment through a pointer that could refer to x . For example, the assignment $*q:=y$ is a definition of x if it is possible that q points to x . Methods for determining what a pointer could point to are also discussed in Section 10.8, but in the absence of any knowledge to the contrary, we must assume that an assignment through a pointer is a definition of every variable.

We say a definition d *reaches* a point p if there is a path from the point immediately following d to p , such that d is not "killed" along that path. Intuitively, if a definition d of some variable a reaches point p , then d might be the place at which the value of a used at p might last have been defined. We *kill* a definition of a variable a if between two points along the path there is a definition of a . Notice that only unambiguous definitions of a kill other definitions of a . Thus, a point can be reached by an unambiguous definition and an ambiguous definition of the same variable appearing later along one path.

For example, both the definitions $i:=m-1$ and $j:=n$ in block B_1 in Fig. 10.19 reach the beginning of block B_2 , as does the definition $j:=j-1$, provided there are no assignments to or reads of j in B_4 , B_5 , or the portion of B_3 following that definition. However, the assignment to j in B_3 kills the definition $j:=n$, so the latter does not reach B_4 , B_5 , or B_6 .

By defining reaching definitions as we have, we sometimes allow inaccuracies. However, they are all in the "safe," or "conservative" direction. For example, notice our assumption that all edges of a flow graph can be traversed. This may not be true in practice. For example, for no values of a and b can control actually reach the assignment $a:=4$ in the following program fragment:

```

if a = b then a := 2
else if a = b then a := 4
    
```

To decide in general whether each path in a flow graph can be taken is an undecidable problem, and we shall not attempt to solve it.

A recurring theme in the design of code improving transformations is that we must make only conservative decisions in the face of any doubt, although conservative strategies may cause us to miss some transformations that we actually could make safely. A decision is *conservative* if it never leads to a change in what the program computes. In applications of reaching definitions, it is normally conservative to assume that a definition can reach a point even if it might not. Thus, we allow paths that may never be traversed in any execution of the program, and we allow definitions to pass through ambiguous definitions of the same variable.

Data-Flow Analysis of Structured Programs

Flow graphs for control-flow constructs such as do-while statements have a useful property: there is a single beginning point at which control enters and a single end point that control leaves from when execution of the statement is over. We exploit this property when we talk of the definitions reaching the beginning and end of statements with the following syntax

$$\begin{aligned}
 S &\rightarrow \text{id} := E \mid S ; S \mid \text{if } E \text{ then } S \text{ else } S \mid \text{do } S \text{ while } E \\
 E &\rightarrow \text{id} + \text{id} \mid \text{id}
 \end{aligned}$$

Expressions in this language are similar to those in intermediate code, but the flow graphs for statements have restricted forms that are suggested by the diagrams in Fig. 10.20. A primary purpose of this section is to study the data-flow equations summarized in Fig. 10.21.

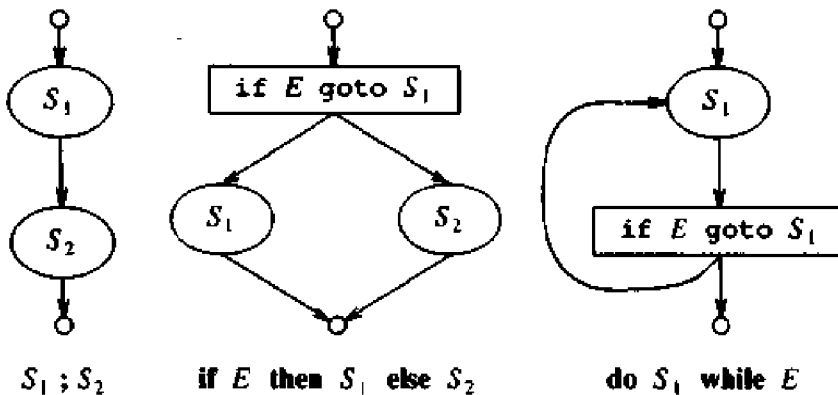


Fig. 10.20. Some structured control constructs.

We define a portion of a flow graph called a *region* to be a set of nodes N that includes a *header*, which dominates all other nodes in the region. All

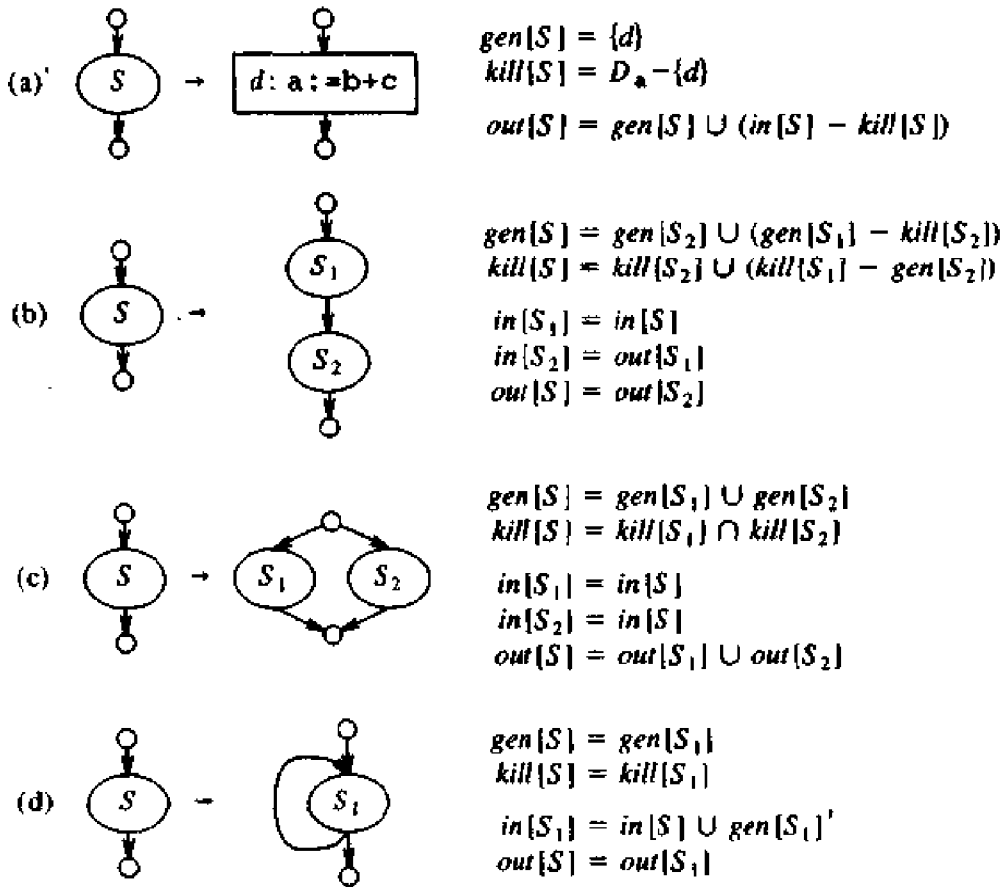


Fig. 10.21. Data-flow equations for reaching definitions.

edges between nodes in N are in the region, except (possibly) for some that enter the header.⁴ The portion of a flow graph corresponding to a statement S is a region that obeys the further restriction that control can flow to just one outside block when it leaves the region.

As a technical convenience we assume that there are dummy blocks with no statements (indicated by open circles in Fig. 10.20) through which control flows just before it enters and just before it leaves the region. We say that the beginning points of the dummy blocks at the entry and exit of a statement's region are the *beginning* and *end* points, respectively, of the statement.

The equations in Fig. 10.21 are an inductive, or syntax-directed, definition of the sets $\text{in}[S]$, $\text{out}[S]$, $\text{gen}[S]$, and $\text{kill}[S]$ for all statements S . Sets $\text{gen}[S]$ and $\text{kill}[S]$ are synthesized attributes; they are computed bottom-up, from the smallest statements to the largest. Our desire is that definition d is in $\text{gen}[S]$

⁴ A loop is a special case of a region that is strongly connected and includes all its back edges into the header.

if d reaches the end of S , independently of whether it reaches the beginning of S . Put another way, d must appear in S and reach the end of S via a path that does not go outside S . That is the justification for saying that $gen[S]$ is the set of definitions "generated by S ."

Similarly, we intend that $kill[S]$ be the set of definitions that never reach the end of S , even if they reach the beginning. Thus, it makes sense to view these definitions as "killed by S ." In order for definition d to be in $kill[S]$, every path from the beginning to the end of S must have an unambiguous definition of the same variable defined by d , and if d appears in S , then following every occurrence of d along any path must be another definition of the same variable.⁵

The rules for gen and $kill$, being synthesized translations, are relatively easy to understand. To begin, observe the rules in Fig. 10.21(a) for a single assignment of variable a . Surely that assignment is a definition of a , say definition d . Then d is the only definition sure to reach the end of the statement regardless of whether it reaches the beginning. Thus

$$gen[S] = \{d\}$$

On the other hand, d "kills" all other definitions of a , so we write

$$kill[S] = D_a - \{d\}$$

where D_a is the set of all definitions in the program for variable a .

The rule for a cascade of statements, illustrated in Fig. 10.21(b), is a bit more subtle. Under what circumstances is definition d generated by $S = S_1 ; S_2$? First of all, if it is generated by S_2 , then it is surely generated by S . If d is generated by S_1 , it will reach the end of S provided it is not killed by S_2 . Thus, we write

$$gen[S] = gen[S_2] \cup (gen[S_1] - kill[S_2])$$

Similar reasoning applies to the killing of a definition, so we have

$$kill[S] = kill[S_2] \cup (kill[S_1] - gen[S_2])$$

For the if-statement, illustrated in Fig. 10.21(c), we note that if either branch of the "if" generates a definition, then that definition reaches the end of the statement S . Thus,

$$gen[S] = gen[S_1] \cup gen[S_2]$$

However, in order to "kill" definition d , the variable defined by d must be killed along any path from the beginning to the end of S . In particular, it must be killed along either branch, so

$$kill[S] = kill[S_1] \cap kill[S_2]$$

⁵ In this introductory section, we are going to assume that all definitions are unambiguous. Section 10.8 deals with the modifications necessary to handle ambiguous definitions.

Lastly, consider the rules for loops, in Fig. 10.21(d). Simply stated, the loop does not affect *gen* or *kill*. If definition d is generated within S_1 , then it reaches both the end of S_1 and the end of S . Conversely, if d is generated within S , it can only be generated within S_1 . If d is killed by S_1 , then going around the loop will not help; the variable of d gets redefined within S_1 each time around. Conversely, if d is killed by S , then it must surely be killed by S_1 . We conclude that

$$\begin{aligned} \text{gen}[S] &= \text{gen}[S_1] \\ \text{kill}[S] &= \text{kill}[S_1] \end{aligned}$$

Conservative Estimation of Data-Flow Information

There is a subtle miscalculation in the rules for *gen* and *kill* given in Fig. 10.21. We have made the assumption that the conditional expression E in the if and do statements are "uninterpreted;" that is, there exist inputs to the program that make their branches go either way. Put another way, we assume that any graph-theoretic path in the flow diagram is also an *execution path*, i.e., a path that is executed when the program is run with at least one possible input.

That is not always the case, and in fact we cannot decide in general whether a branch can be taken. Suppose, for example, that the expression E in an if-statement were always true. Then the path through S_2 in Fig. 10.21(c) could never be taken. This has two consequences. First, a definition generated by S_2 is not really generated by S , because there is no way to get from the beginning of S into the statement S_2 . Second, no definition in $\text{kill}[S_1]$ can reach the end of S . Therefore, each such definition should logically be in $\text{kill}[S]$, even if it is not in $\text{kill}[S_2]$.

When we compare the computed *gen* with the "true" *gen* we discover that the true *gen* is always a subset of the computed *gen*. On the other hand, the true *kill* is always a superset of the computed *kill*. These containments hold even after we consider the other rules in Fig. 10.21. For example, if the expression E in a do- S -while- E statement can never be false, then we can never get out of the loop. Thus, the true *gen* is $\emptyset$, and every definition is "killed" by the loop. The case of a cascade of statements, in Fig. 10.21(b), where the inability to get out of S_1 or S_2 because of an infinite loop must be taken into account, is left as an exercise.

It is natural to wonder whether these differences between the true and computed *gen* and *kill* sets present a serious obstacle to data-flow analysis. The answer lies in the use intended for these data. In the particular case of reaching definitions, we normally use the information to infer that the value of a variable x at a point is limited to some small number of possibilities. For example, if we find that the only definitions of x reaching that point are of the form $x := 1$, we may infer that x has the value 1 at the point. Thus, we might decide to replace references to x by references to 1.

As a consequence, overestimating the set of definitions reaching a point

does not seem serious; it merely stops us from doing an optimization that we could legitimately do. On the other hand, underestimating the set of definitions is a fatal error; it could lead us into making a change in the program that changes what the program computes. For example, we may think all reaching definitions of x give x the value 1, and so replace x by 1; but there is another undetected reaching definition that gives x the value 2. For the case of reaching definitions, then, we call a set of definitions *safe* or *conservative* if the estimate is a superset (not necessarily a proper superset) of the true set of reaching definitions. We call the estimate *unsafe* if it is not necessarily a superset of the truth.

For each data-flow problem we must examine the effect of inaccurate estimates on the kinds of program changes they may cause. We generally accept discrepancies that are *safe* in the sense that they may forbid optimizations that could legally be made, but not accept discrepancies that are *unsafe* in the sense that they may cause "optimizations" that do not preserve the externally observed behavior of the program. In each data-flow problem, either a subset or a superset of the true answer (but not both) is usually safe.

Returning now to the implications of safety on the estimation of *gen* and *kill* for reaching definitions, note that our discrepancies, supersets for *gen* and subsets for *kill* are both in the safe direction. Intuitively, increasing *gen* adds to the set of definitions that can reach a point, and cannot prevent a definition from reaching a place that it truly reached. Likewise, decreasing *kill* can only increase the set of definitions reaching any given point.

Computation of *in* and *out*

Many data-flow problems can be solved by synthesized translations similar to those used to compute *gen* and *kill*. For example, we may wish to determine, for each statement S , the set of variables that are defined within S . This information can be computed by equations analogous to those for *gen*, without even requiring sets analogous to *kill*. It can be used, for example, to determine loop-invariant computations.

However, there are other kinds of data-flow information, such as the reaching-definitions problem we have used as an example, where we also need to compute certain inherited attributes. It turns out that *in* is an inherited attribute, and *out* is a synthesized attribute depending on *in*. We intend that $in[S]$ be the set of definitions reaching the beginning of S , taking into account the flow of control throughout the entire program, including statements outside of S or within which S is nested. The set $out[S]$ is defined similarly for the end of S . It is important to note the distinction between $out[S]$ and $gen[S]$. The latter is the set of definitions that reach the end of S without following paths outside S .

As a simple example of the difference, consider the cascade of statements in Fig. 10.21(b). A statement d may be generated in S_1 and therefore reach the beginning of S_2 . If d is not in $kill[S_2]$, d will reach the end of S_2 , and

therefore be in $out[S_2]$. However, d is not in $gen[S_2]$.

After computing $gen[S]$ and $kill[S]$ bottom-up, for all statements S , we may compute in and out starting at the statement representing the complete program, understanding that $in[S_0] = \emptyset$ if S_0 is the complete program. That is, no definitions reach the beginning of the program. For each of the four types of statements in Fig. 10.21, we may assume that $in[S]$ is known. We must use it to compute in for each of the substatements of S [which is trivial in cases (b)–(d) and irrelevant in case (a)]. Then, we recursively (top-down) compute out for each of the substatements S_1 or S_2 , and use these sets to compute $out[S]$.

The simplest case is Fig. 10.21(a), where the statement is an assignment. Assuming we know $in[S]$, we compute out by equation (10.5), that is

$$out[S] = gen[S] \cup (in[S] - kill[S])$$

In words, a definition reaches the end of S if either it is generated by S (i.e., it is the definition d that is the statement), or it reaches the beginning of the statement and is not killed by the statement.

Suppose we have computed $in[S]$ and S is the cascade of two statements $S_1; S_2$, as in the second case of Fig. 10.21. We start by observing $in[S_1] = in[S]$. Then, we recursively compute $out[S_1]$, which gives us $in[S_2]$, since a definition reaches the beginning of S_2 if and only if it reaches the end of S_1 . Now we can recursively compute $out[S_2]$, and this set is equal to $out[S]$.

Next, consider the if-statement of Fig. 10.21(c). As we have conservatively assumed that control can follow either branch, a definition reaches the beginning of S_1 or S_2 exactly when it reaches the beginning of S . That is,

$$in[S_1] = in[S_2] = in[S]$$

It also follows from the diagram in Fig. 10.21(c) that a definition reaches the end of S if and only if it reaches the end of one or both substatements; i.e.,

$$out[S] = out[S_1] \cup out[S_2]$$

We thus may use these equations to compute $in[S_1]$ and $in[S_2]$ from $in[S]$, recursively compute $out[S_1]$ and $out[S_2]$, then use these to compute $out[S]$.

Dealing With Loops

The last case, Fig. 10.21(d), presents special problems. Let us again assume we are given $gen[S_1]$ and $kill[S_1]$, having computed them bottom-up, and let us assume we are given $in[S]$, as we are in the process of performing a depth-first traversal of the parse tree. Unlike cases (b) and (c), we cannot simply use $in[S]$ as $in[S_1]$, because definitions inside S_1 that reach the end of S_1 are able to follow the arc back to the beginning of S_1 , and therefore these definitions also are in $in[S_1]$. Rather, we have

$$in[S_1] = in[S] \cup out[S_1] \tag{10.6}$$

We also have the obvious equation for $out[S]$:

$$out[S] = out[S_1]$$

which we can use once we have computed $out[S_1]$. However, it seems we cannot compute $in[S_1]$ by (10.6) until we have computed $out[S_1]$, and our general plan has been to compute out for a statement by first computing in for that statement.

Fortunately, there is a direct way to express out in terms of in ; it is given by (10.5), or in this particular case:

$$out[S_1] = gen[S_1] \cup (in[S_1] - kill[S_1]) \quad (10.7)$$

It is important to understand what is going on here. We do not really know that (10.7) is true about an arbitrary statement S_1 ; we only suspect that it should be true because it "makes sense" that a definition should reach the end of a statement if and only if it is either generated inside the statement or it reaches the beginning and is not killed. However, the only way we know to compute out for a statement is by the equations given in Fig. 10.21(a)–(c). We are going to assume (10.7) and derive the equations for in and out in Fig. 10.21(d). Then, we can use the equations of Fig. 10.21(a)–(d) to prove that (10.7) holds for an arbitrary S_1 . We could then put these proofs together to make a valid proof by induction on the size of a statement S that the equations (10.7) and all the equations of Fig. 10.21 hold for S and all its substatements. We shall not do so; we leave the proofs as an exercise, but the reasoning we do follow here should prove instructive.

Even assuming (10.6) and (10.7) we are still not out of the woods. These two equations define a recurrence for $in[S_1]$ and $out[S_1]$ simultaneously. Let us write the equations as

$$\begin{aligned} I &= J \cup O \\ O &= G \cup (I - K) \end{aligned} \quad (10.8)$$

where I , O , J , G , and K correspond to $in[S_1]$, $out[S_1]$, $in[S]$, $gen[S_1]$, and $kill[S_1]$, respectively. The first two are variables; the other three are constants.

To solve (10.8), let us assume that $O = \emptyset$. We could then use the first equation in (10.8) to compute an estimate of I , that is,

$$I^1 = J$$

Next, we can use the second equation to get a better estimate of O :

$$O^1 = G \cup (I^1 - K) = G \cup (J - K)$$

Applying the first equation to this new estimate of O gives us:

$$I^2 = J \cup O^1 = J \cup G \cup (J - K) = J \cup G$$

If we then reapply the second equation, the next estimate of O is:

$$O^2 = G \cup (I^2 - K) = G \cup (J \cup G - K) = G \cup (J - K)$$

Note that $O^2 = O^1$. Thus, if we compute the next estimate of I , it will be equal to I^1 , which will give us another estimate of O equal to O^1 , and so on. Thus, the limiting values for I and O are those given for I^1 and O^1 above. We have thus derived the equations in Fig. 10.21(d), which are

$$\begin{aligned} in[S_1] &= in[S] \cup gen[S_1] \\ out[S] &= out[S_1] \end{aligned}$$

The first of these equations is from the calculation above; the second follows from examination of the graph in Fig. 10.21(d).

A detail that remains is why we were entitled to start with the estimate $O = \emptyset$. Recall in our discussion of conservative estimates we suggested that sets like $out[S_1]$, which O stands for, should be overestimated rather than underestimated. In fact, if we were to start with $O = \{d\}$, where d is a definition appearing neither in J , G , or K , then d would wind up in the limiting values of both I and O .

Here, we must invoke the intended meanings of *in* and *out*. If such a d really belonged in $in[S_1]$, there would have to be a path from wherever d is located to the beginning of S_1 that showed how d reaches that point. If d is outside S , then d would have to be in $in[S]$, while if d were inside S (and therefore inside S_1) it would have to be in $gen[S_1]$. In the first case, d is in J and therefore placed in I by (10.8). In the second case, d is in G and again transmitted to I via O in (10.8). The conclusion is that starting with a too small estimate and building upward by adjoining more definitions to I and O is a safe way to estimate $in[S_1]$.

Representation of Sets

Sets of definitions, such as $gen[S]$ and $kill[S]$, can be represented compactly using bit vectors. We assign a number to each definition of interest in the flow graph. Then the bit vector representing a set of definitions will have 1 in position i if and only if the definition numbered i is in the set.

The number of a definition statement can be taken as the index of the statement in an array holding pointers to statements. However, not all definitions may be of interest during global data-flow analysis. For example, definitions of temporaries that are used only within a single block, as most temporaries generated for expression evaluation will be, need not be assigned numbers. Therefore, the numbers of definitions of interest will typically be recorded in a separate table.

A bit-vector representation for sets also allows set operations to be implemented efficiently. The union and intersection of two sets can be implemented by logical **or** and logical **and**, respectively, basic operations in most systems-oriented programming languages. The difference $A - B$ of sets A and B can be implemented by taking the complement of B and then using logical **and** to compute $A \wedge \neg B$.

Example 10.13. Figure 10.22 shows a program with seven definitions.

```

/* d1 */   i := m-1;
/* d2 */   j := n;
/* d3 */   a := u1;
           do
/* d4 */       i := i+1;
/* d5 */       j := j-1;
           if e1 then
/* d6 */       a := u2
           else
/* d7 */       i := u3
           while e2

```

Fig. 10.22. Program for illustrating reaching definitions.

indicated by d_1 through d_7 in the comments to the left of the definitions. Bit vectors representing the *gen* and *kill* sets for the statements in Fig. 10.22 are shown to the left of syntax-tree nodes in Fig. 10.23. The sets themselves were computed by applying the data-flow equations in Fig. 10.21 to the statements represented by the syntax-tree nodes.

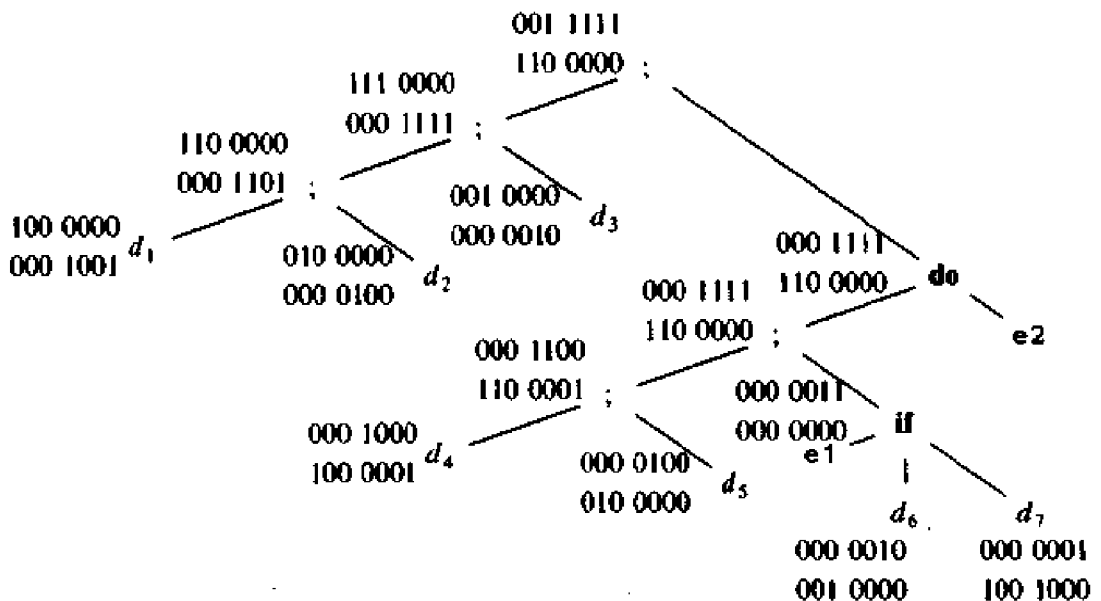


Fig. 10.23. The *gen* and *kill* sets at nodes in a syntax tree.

Consider the node for d_7 in the bottom-right corner of Fig. 10.23. The *gen* set $\{d_7\}$ is represented by 000 0001 and the *kill* set $\{d_1, d_4\}$ by 100 1000. That is, d_7 kills all the other definitions of i , its variable.

The second and third children of the **if** node represent the then and else parts, respectively, of the conditional. Note that the *gen* set 0000011 at the **if** node is the union of the sets 0000010 and 0000001 at the second and third children. The *kill* set is empty because the definitions killed by the then and else parts are disjoint.

Data-flow equations for a cascade of statements are applied at the parent of the **if** node. The *kill* set at this node is obtained by

$$0000000 \cup (1100001 - 0000011) = 1100000$$

In words, nothing is killed by the conditional, and d_7 , killed by the statement d_4 , is generated by the conditional, so only d_1 and d_2 are in the *kill* set of the parent of the **if** node.

Now we can compute *in* and *out*, starting from the top of the parse tree. We assume the *in* set at the root of the syntax tree is empty. Thus *out* for the left child of the root is *gen* of that node, or 1110000. This is also the value of the *in* set at the **do** node. From the data-flow equations associated with the **do** production in Fig. 10.21, the *in* set for the statement inside the **do** loop is obtained by taking the union of the *in* set 1110000 at the **do** node and the *gen* set 0001111 at the statement. The union yields 1111111, so all definitions can reach the beginning of the loop body. However, at the point just before definition d_3 , the *in* set is 0111110, since definition d_4 kills d_1 and d_7 . The balance of the *in* and *out* calculations are left as an exercise. $\square$

Local Reaching Definitions

Space for data-flow information can be traded for time, by saving information only at certain points and, as needed, recomputing information at intervening points. Basic blocks are usually treated as a unit during global flow analysis, with attention restricted to only those points that are the beginnings of blocks. Since there are usually many more points than blocks, restricting our effort to blocks is a significant savings. When needed, the reaching definitions for all points in a block can be calculated from the reaching definitions for the beginning of a block.

In more detail, consider a sequence of assignments $S_1; S_2; \dots; S_n$ in a basic block B . We refer to the beginning of B as point p_0 , to the point between statements S_i and S_{i+1} as p_i , and to the end of the block as point p_n . The definitions reaching point p_j can be obtained from $in[B]$ by considering the statements $S_1; S_2; \dots; S_j$, starting with S_1 and applying the data-flow equations in Fig. 10.21 for cascades of statements. Initially, let $D = in[B]$. When S_i is considered, we delete from D the definitions killed by S_i and add the definitions generated by S_i . At the end, D consists of the definitions reaching p_j .

Use-Definition Chains

It is often convenient to store the reaching definition information as “use-definition chains” or “ud-chains,” which are lists, for each use of a variable, of all the definitions that reach that use. If a use of variable a in block B is preceded by no unambiguous definition of a , then the ud-chain for that use of a is the set of definitions in $in[B]$ that are definitions of a . If there are unambiguous definitions of a within B preceding this use of a , then only the last such definition of a will be on the ud-chain, and $in[B]$ is not placed on the ud-chain. In addition, if there are ambiguous definitions of a , then all of these for which no unambiguous definition of a lies between it and the use of a are on the ud-chain for this use of a .

Evaluation Order

The techniques for conserving space during attribute evaluation, discussed in Chapter 5, also apply to the computation of data-flow information using specifications like the one in Fig. 10.21. Specifically, the only constraint on the evaluation order for the *gen*, *kill*, *in*, and *out* sets for statements is that imposed by dependencies between these sets. Having chosen an evaluation order, we are free to release the space for a set after all uses of it have occurred.

The data-flow equations of this section differ in one respect from semantic rules for attributes in Chapter 5: circular dependencies between attributes were not allowed in Chapter 5, but we have seen that data-flow equations may have circular dependencies, e.g., $in[S_1]$ and $out[S_1]$ depend on each other in 10.8. For the reaching-definitions problem, the data-flow equations can be rewritten to eliminate the circularity — compare the noncircular equations in Fig. 10.21 with 10.8. Once a noncircular specification is obtained, the techniques of Chapter 5 can be applied to obtain efficient solutions of data-flow equations.

General Control Flow

Data-flow analysis must take all control paths into account. If the control paths are evident from the syntax, then data-flow equations can be set up and solved in a syntax-directed manner, as in this section. When programs can contain goto statements, or even the more disciplined break and continue statements, the approach we have taken must be modified to take the actual control paths into account.

Several approaches may be taken. The iterative method in the next section works for arbitrary flow graphs. Since the flow graphs obtained in the presence of break and continue statements are reducible, such constructs can be handled systematically using the interval-based methods to be discussed in Section 10.10.

However, the syntax-directed approach need not be abandoned when break and continue statements are allowed. Before leaving this section we consider an example suggesting how break-statements can be accommodated, leaving the development of the ideas to Section 10.10.

Example 10.14. The break-statement within the do-while loop in Fig. 10.24 is equivalent to a jump to the end of the loop. How then are we to define the *gen* set for the following statement?

```
if e3 then a := u2
else begin i := u3; break end
```

We define the *gen* set to be $\{d_6\}$, where d_6 is the definition $a := u2$, because d_6 is the only definition generated along the control paths from the beginning to the end points of the statement. Definition d_7 , i.e., $i := u3$, will get taken into account when the entire do-while loop is treated.

```
/* d1 */   i := m-1;
/* d2 */   j := n;
/* d3 */   a := u1;
           do
/* d4 */       i := i+1;
/* d5 */       j := j-1;
               if e3 then
/* d6 */         a := u2
               else begin
/* d7 */           i := u3;
                   break
               end
           while e2
```

Fig. 10.24. Program containing a break-statement.

There is a programming trick that allows us to ignore the jump caused by the break-statement while we are processing the statements within the body of the loop: we take the *gen* and *kill* sets for a break-statement to be the empty set and U , the universal set of all definitions, respectively, as shown in Fig. 10.25. The remaining *gen* and *kill* sets shown in Fig. 10.25 are determined using the data-flow equations in Fig. 10.21, with the *gen* set shown above the *kill* set. Statements S_1 and S_2 represent sequences of assignments. The *gen* and *kill* sets at the do-node remain to be determined.

The end point of any sequence of statements ending in a break cannot be reached, so there is no harm in taking the *gen* set for the sequence to be $\emptyset$ and the *kill* set to be U ; the result will still be a conservative estimate of *in* and *out*. Similarly, the end point of the if-statement can only be reached through the then part, and the *gen* and *kill* sets at the if node in Fig. 10.25 are indeed the same as those at its second child.

10.6 ITERATIVE SOLUTION OF DATA-FLOW EQUATIONS

The method of the last section is simple and efficient when it applies, but for languages like Fortran or Pascal that permit arbitrary flow graphs, it is not sufficiently general. Section 10.10 discusses "interval analysis," a way to obtain the advantages of the syntax-directed approach to data-flow analysis for general flow graphs, at the expense of considerably more conceptual complexity.

Here, we shall discuss another important approach to solving data-flow problems. Instead of trying to use the parse tree to drive the calculation of *in* and *out* sets, we first build the flow graph, and then compute *in* and *out* for each node simultaneously. While discussing this new method, we shall also take the opportunity to introduce the reader to many different data-flow analysis problems, show some of their applications, and point out the differences among the problems.

The equations for many data-flow problems are similar in form in that information is being "generated" and "killed." However, there are two principal ways in which equations differ in detail.

1. The equations in the last section for reaching definitions are *forward* equations in the sense that the *out* sets are computed in terms of the *in* sets. We shall also see data-flow problems that are *backward* in that the *in* sets are computed from the *out* sets.
2. When there is more than one edge entering block *B*, the definitions reaching the beginning of *B* are the union of the definitions arriving along each of the edges. We therefore say that union is the *confluence operator*. In contrast, we shall consider problems like global available expressions, where intersection is the confluence operator, because an expression is only available at the beginning of *B* if it is available at the end of every predecessor of *B*. In Section 10.11 we shall see other examples of confluence operators.

In this section we shall see examples of both forward and backward equations with union and intersection taking turns as the confluence operator.

Iterative Algorithm for Reaching Definitions

For each basic block *B*, we can define *out*[*B*], *gen*[*B*], *kill*[*B*], and *in*[*B*], as in the last section, by noting that each block *B* may be viewed as a statement that is the cascade of one or more assignment statements. Assuming that *gen* and *kill* have been computed for each block, we can create two groups of equations, shown in (10.9) below, that relate *in* and *out*. The first group of equations follows from the observation that *in*[*B*] is the union of the definitions arriving from all predecessors of *B*. The second group are special cases of the general law (10.5) that we claim holds for all statements. These two groups are:

$$\begin{aligned}
 in[B] &= \bigcup_{P \text{ a predecessor of } B} out[P] \\
 out[B] &= gen[B] \cup (in[B] - kill[B])
 \end{aligned}
 \tag{10.9}$$

If a flow graph has n basic blocks, we get $2n$ equations from (10.9). The $2n$ equations can be solved by treating them as recurrences for computing the *in* and *out* sets, just as the data-flow equations (10.6) and (10.5) for **do-while** statements were solved in the last section. In the last section we began with the empty set of definitions to be the starting estimate for all the *out* sets. Here we shall start with empty *in* sets, since we note from (10.9) that the *in* sets, being the union of *out* sets, will be empty if the *out* sets are. While we were able to agree that the equations (10.6) and (10.7) needed only one iteration, in the case of these more complex equations, we cannot *a priori* bound the number of iterations.

Algorithm 10.2. Reaching definitions.

Input. A flow graph for which $kill[B]$ and $gen[B]$ have been computed for each block B .

Output. $in[B]$ and $out[B]$ for each block B .

Method. We use an iterative approach, starting with the "estimate" $in[B] = \emptyset$ for all B and converging to the desired values of *in* and *out*. As we must iterate until the *in*'s (and hence the *out*'s) converge, we use a boolean variable *change* to record on each pass through the blocks whether any *in* has changed. The algorithm is sketched in Fig. 10.26. $\square$

```

/* initialize out on the assumption in[B] = ∅ for all B */
(1) for each block B do out[B] := gen[B];
(2) change := true; /* to get the while-loop going */
(3) while change do begin
(4)     change := false;
(5)     for each block B do begin
(6)         in[B] :=  $\bigcup_{P \text{ a predecessor of } B} out[P]$ ;
(7)         oldout := out[B];
(8)         out[B] := gen[B]  $\cup$  (in[B] - kill[B]);
(9)         if out[B]  $\neq$  oldout then change := true
    end
end

```

Fig. 10.26. Algorithm to compute *in* and *out*.

Intuitively, Algorithm 10.2 propagates definitions as far as they will go without being killed, in a sense simulating all possible executions of the

program. The bibliographic notes contain references where formal proofs of the correctness of this and other data-flow analysis problems can be found.

We can see that the algorithm will eventually halt because $out[B]$ never decreases in size for any B ; once a definition is added, it stays there forever. (The proof of this fact is left as an inductive exercise.) Since the set of all definitions is finite, there must eventually be a pass of the while-loop in which $oldout = out[B]$ for each B at line (9). Then $change$ will remain false and the algorithm terminates. We are safe terminating then because if the out 's have not changed, the in 's will not change on the next pass. And, if the in 's do not change, the out 's cannot, so on all subsequent passes there can be no changes.

It may be shown that an upper bound on the number of times around the while-loop is the number of nodes in the flow graph. Intuitively, the reason is that if a definition reaches a point, it can do so along a cycle-free path, and the number of nodes in a flow graph is an upper bound on the number of nodes in a cycle-free path. Each time around the while-loop, the definition progresses by at least one node along the path in question.

In fact, if we properly order the blocks in the for-loop of line (5), there is empirical evidence that the average number of iterations on real programs is under 5 (see Section 10.10). Since the sets can be represented by bit vectors, and the operations on these sets can be implemented by logical operations on the bit vectors, Algorithm 10.2 is surprisingly efficient in practice.

Example 10.15. The flow graph of Fig. 10.27 has been derived from the program in Fig. 10.22 of the last section. We shall apply Algorithm 10.2 to this flow graph so the approaches of the two sections can be compared.

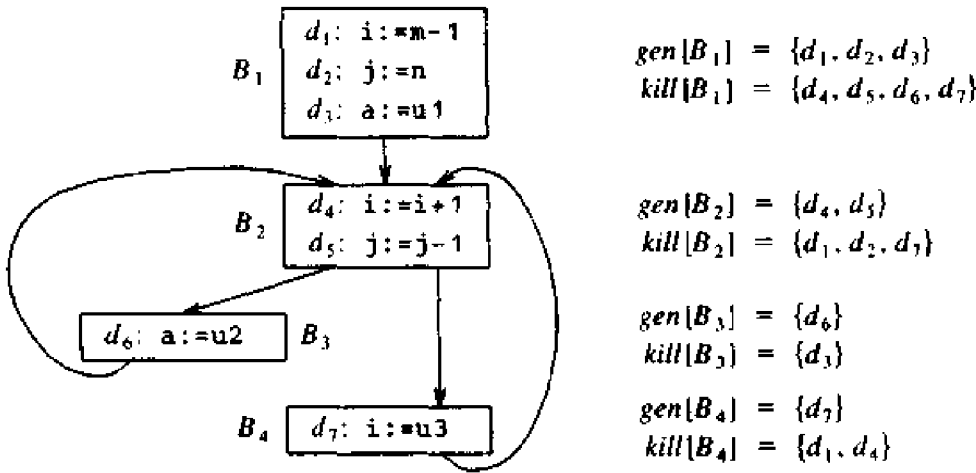


Fig. 10.27. Flow graph for illustrating reaching definitions.

Only the definitions $d_1, d_2, \dots, d_7$ defining i, j , and a in Fig. 10.27 are of interest. As in the last section, we shall represent sets of definitions by bit vectors, where bit i from the left represents definition d_i .

The loop of line (1) in Fig. 10.26 initializes $out[B] = gen[B]$ for each B ,

and these initial values of $out[B]$ are shown in the table in Fig. 10.28. The initial values ($\emptyset$) of each $in[B]$ are not computed or used, but are shown for completeness. Suppose the for-loop of line (5) is executed with $B = B_1, B_2, B_3, B_4$, in that order. With $B = B_1$, there are no predecessors for the initial node so $in[B_1]$ remains the empty set, represented by 000 0000; as a result, $out[B_1]$ remains equal to $gen[B_1]$. This value does not differ from $oldout$ computed at line (7), so we do not yet set $change$ to **true**.

Then we consider $B = B_2$ and compute

$$\begin{aligned} in[B_2] &= out[B_1] \cup out[B_3] \cup out[B_4] \\ &= 111\,0000 + 000\,0010 + 000\,0001 = 111\,0011 \\ out[B_2] &= gen[B_2] \cup (in[B_2] - kill[B_2]) \\ &= 000\,1100 + (111\,0011 - 110\,0001) = 001\,1110 \end{aligned}$$

This computation is summarized in Fig. 10.28. At the end of the first pass, $out[B_4] = 001\,0111$, reflecting the fact that d_7 is generated and that d_3, d_5 , and d_6 reach B_4 and are not killed in B_4 . From the second pass on there are no changes in any of the out sets, so the algorithm terminates. $\square$

BLOCK B	INITIAL		PASS 1		PASS 2	
	$in[B]$	$out[B]$	$in[B]$	$out[B]$	$in[B]$	$out[B]$
B_1	000 0000	111 0000	000 0000	111 0000	000 0000	111 0000
B_2	000 0000	000 1100	111 0011	001 1110	111 1111	001 1110
B_3	000 0000	000 0010	001 1110	000 1110	001 1110	000 1110
B_4	000 0000	000 0001	001 1110	001 0111	001 1110	001 0111

Fig. 10.28. Computation of in and out .

Available Expressions

An expression $x+y$ is *available* at a point p if every path (not necessarily cycle-free) from the initial node to p evaluates $x+y$, and after the last such evaluation prior to reaching p , there are no subsequent assignments to x or y . For available expressions we say that a block *kills* expression $x+y$ if it assigns (or may assign) x or y and does not subsequently recompute $x+y$. A block *generates* expression $x+y$ if it definitely evaluates $x+y$ and does not subsequently redefine x or y .

Note that the notion of “killing” or “generating” an available expression is not exactly the same as that for reaching definitions. Nevertheless, these notions of “kill” and “generate” obey the same laws as they do for reaching definitions. We could compute them exactly as we did in Section 10.5, provided we modified the rules in 10.21(a) for a simple assignment statement.

The primary use of available expressions information is for detecting common subexpressions. For example, in Fig. 10.29, the expression $4*i$ in block B_3 will be a common subexpression if $4*i$ is available at the entry point of block B_3 . It will be available if i is not assigned a new value in block B_2 , or if, as in Fig. 10.29(b), $4*i$ is recomputed after i is assigned in B_2 .

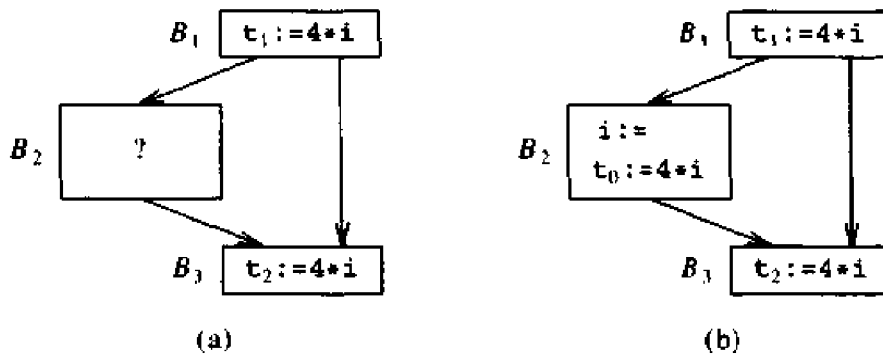


Fig. 10.29. Potential common subexpressions across blocks.

We can easily compute the set of generated expressions for each point in a block, working from beginning to end of the block. At the point prior to the block, assume no expressions are available. If at point p set A of expressions is available, and q is the point after p , with statement $x := y + z$ between them, then we form the set of expressions available at q by the following two steps.

1. Add to A the expression $y + z$.
2. Delete from A any expression involving x .

Note the steps must be done in the correct order, as x could be the same as y or z . After we reach the end of the block, A is the set of generated expressions for the block. The set of killed expressions is all expressions, say, $y + z$ such that either y or z is defined in the block, and $y + z$ is not generated by the block.

Example 10.16. Consider the four statements of Fig. 10.30. After the first, $b + c$ is available. After the second, $a - d$ becomes available, but $b + c$ is no longer available, because b has been redefined. The third does not make $b + c$ available again, because the value of c is immediately changed. After the last statement, $a - d$ is no longer available, because d has changed. Thus no statements are generated, and all statements involving a , b , c , or d are killed. $\square$

We can find available expressions in a manner reminiscent of the way reaching definitions are computed. Suppose U is the "universal" set of all expressions appearing on the right of one or more statements of the program. For each block B , let $in[B]$ be the set of expressions in U that are available at the point just before the beginning of B . Let $out[B]$ be the same for the point following the end of B . Define $e_gen[B]$ to be the expressions generated by B

STATEMENTS	AVAILABLE EXPRESSIONS
.....	none
a := b+c	 only b+c
b := a-d	 only a-d
c := b+c	 only a-d
d := a-d	 none

Fig. 10.30. Computation of available expressions.

and $e_kill[B]$ to be the set of expressions in U killed in B . Note that in , out , e_gen , and e_kill can all be represented by bit vectors. The following equations relate the unknowns in and out to each other and the known quantities e_gen and e_kill .

$$\begin{aligned}
 out[B] &= e_gen[B] \cup (in[B] - e_kill[B]) \\
 in[B] &= \bigcap_{\substack{P \text{ a prede-} \\ \text{cessor of } B}} out[P] \quad \text{for } B \text{ not initial} \\
 in[B_1] &= \emptyset \quad \text{where } B_1 \text{ is the initial block.}
 \end{aligned} \tag{10.10}$$

Equations (10.10) look almost identical to equations (10.9) for reaching definitions. The first difference is that in for the initial node is handled as a special case. This is justified on the grounds that nothing is available if the program has just begun at the initial node, even though some expression might be available along all paths to the initial node from elsewhere in the program. If we did not force $in[B_1]$ to be empty, we might erroneously deduce that certain expressions were available before the program started.

The second, and more important, difference is that the confluence operator is intersection rather than union. This operator is the proper one because an expression is available at the beginning of a block only if it is available at the end of all its predecessors. In contrast, a definition reaches the beginning of a block whenever it reaches the end of one or more of its predecessors.

The use of $\cap$ rather than $\cup$ makes equations (10.10) behave differently from (10.9). While neither set has a unique solution, for (10.9) it is the smallest solution that corresponds to the definition of "reaching," and we obtained that solution by starting with the assumption that nothing reached anywhere, and building up to the solution. In that way, we never assumed that a definition d could reach a point p unless an actual path propagating d to p could be found. In contrast, for equations (10.10) we want the largest possible solution, so we start with an approximation that is too large and work down.

It may not be obvious that by starting with the assumption “everything, i.e., the set U , is available everywhere” and eliminating only those expressions for which we can discover a path along which it is not available, we do reach a set of truly available expressions. In the case of available expressions, it is conservative to produce a subset of the exact set of available expressions, and this is what we do. The argument for subsets being conservative is that our intended use of the information is to replace the computation of an available expression by a previously computed value (see Algorithm 10.5 in the next section), and not knowing an expression is available only inhibits us from changing the code.

Example 10.17. We shall concentrate on a single block, B_2 in Fig. 10.31, to illustrate the effect of the initial approximation of $in[B_2]$ on $out[B_2]$. Let G and K abbreviate $gen[B_2]$ and $kill[B_2]$, respectively. The data-flow equations for block B_2 are:

$$in[B_2] = out[B_1] \cap out[B_2]$$

$$out[B_2] = G \cup (in[B_2] - K)$$

These equations have been rewritten as recurrences in Fig. 10.31, with I^j and O^j being the j th approximations of $in[B_2]$ and $out[B_2]$, respectively. The figure also shows that starting with $I^0 = \emptyset$ we get $O^1 = O^2 = G$, while starting with $I^0 = U$ we get a larger set for O^2 . It so happens that $out[B_2]$ equals O^2 in each case, because the iterations each converge at the points shown.

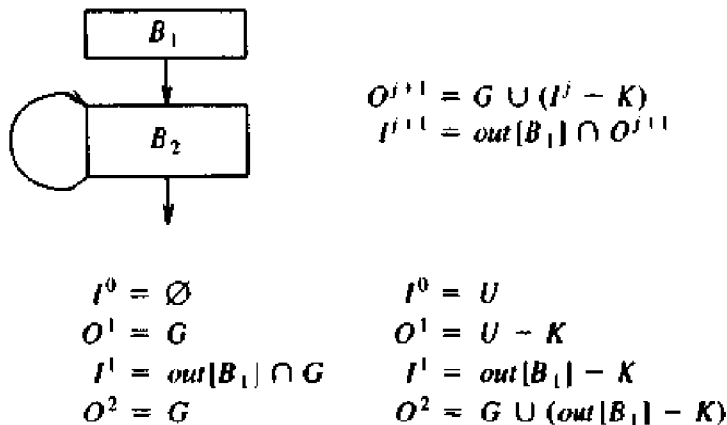


Fig. 10.31. Initializing the in sets to $\emptyset$ is too restrictive.

Intuitively, the solution obtained starting with $I^0 = U$ using

$$out[B_2] = G \cup (out[B_1] - K)$$

is more desirable, because it correctly reflects the fact that expressions in $out[B_1]$ that are not killed by B_2 are available at the end of B_2 , just as the expressions generated by B_2 are. $\square$

Algorithm 10.3. Available expressions.

Input. A flow graph G with $e_kill[B]$ and $e_gen[B]$ computed for each block B . The initial block is B_1 .

Output. The set $in[B]$ for each block B .

Method. Execute the algorithm of Fig. 10.32. The explanation of the steps is similar to that for Fig. 10.26. $\square$

```

 $in[B_1] := \emptyset;$ 
 $out[B_1] := e\_gen[B_1];$  /*  $in$  and  $out$  never change for the initial node,  $B_1$  */
for  $B \neq B_1$  do  $out[B] := U - e\_kill[B];$  /* initial estimate is too large */
 $change := true;$ 
while  $change$  do begin
     $change := false;$ 
    for  $B \neq B_1$  do begin
         $in[B] := \bigcap_{P \text{ a predecessor of } B} out[P];$ 
         $oldout := out[B];$ 
         $out[B] := e\_gen[B] \cup (in[B] - e\_kill[B]);$ 
        if  $out[B] \neq oldout$  then  $change := true$ 
    end
end

```

Fig. 10.32. Available expressions computation.

Live-Variable Analysis

A number of code improving transformations depend on information computed in the direction opposite to the flow of control in a program; we shall consider some of these now. In *live-variable* analysis we wish to know for variable x and point p whether the value of x at p could be used along some path in the flow graph starting at p . If so, we say x is *live* at p ; otherwise x is *dead* at p .

As we saw in Section 9.7, an important use for live-variable information comes when we generate object code. After a value is computed in a register, and presumably used within a block, it is not necessary to store that value if it is dead at the end of the block. Also, if all registers are full and we need another register, we should favor using a register with a dead value, since that value does not have to be stored.

Let us define $in[B]$ to be the set of variables live at the point immediately before block B and define $out[B]$ to be the same at the point immediately after the block. Let $def[B]$ be the set of variables definitely assigned values in B prior to any use of that variable in B , and let $use[B]$ be the set of variables whose values may be used in B prior to any definition of the variable. Then

the equations relating *def* and *use* to the unknowns *in* and *out* are:

$$\begin{aligned} in[B] &= use[B] \cup (out[B] - def[B]) \\ out[B] &= \bigcup_{\substack{S \text{ a suc-} \\ \text{cessor of } B}} in[S] \end{aligned} \quad (10.11)$$

The first group of equations says that a variable is live coming into a block if either it is used before redefinition in the block or it is live coming out of the block and is not redefined in the block. The second group of equations says that a variable is live coming out of a block if and only if it is live coming into one of its successors.

The relation between (10.11) and the reaching-definitions equations (10.9) should be noticed. Here, *in* and *out* have their roles interchanged, and *use* and *def* substitute for *gen* and *kill*, respectively. As for (10.9), the solution to (10.11) is not necessarily unique, and we want the smallest solution. The algorithm used for the minimum solution is essentially a backwards version of Algorithm 10.2. Since the mechanism for detecting changes to any of the *in*'s is so similar to the way we detected changes to *out*'s in Algorithms 10.2 and 10.3, we elide the details of checking for termination.

Algorithm 10.4. Live variable analysis.

Input. A flow graph with *def* and *use* computed for each block.

Output. *out*[*B*], the set of variables live on exit from each block *B* of the flow graph.

Method. Execute the program in Fig. 10.33. □

```

for each block B do in[B] := ∅;
while changes to any of the in's occur do
  for each block B do begin
    out[B] =  $\bigcup_{\substack{S \text{ a suc-} \\ \text{cessor of } B}} in[S]$ 
    in[B] := use[B] ∪ (out[B] - def[B])
  end
end

```

Fig. 10.33. Live variable calculation.

Definition-Use Chains

A calculation done in virtually the same manner as live-variable analysis is *definition-use chaining* (du-chaining). We say a variable is *used* at statement *s* if its *r*-value may be required. For example, *b* and *c* (but not *a*) are used in each of the statements *a* := *b* + *c* and *a*[*b*] := *c*. The du-chaining problem is to

compute for a point p the set of uses s of a variable, say x , such that there is a path from p to s that does not redefine x .

As with live variables, if we can compute $out[B]$, the set of uses reachable from the end of block B , then we can compute the definitions reached from any point p within block B by scanning the portion of block B that follows p . In particular, if there is a definition of variable x in the block, we can determine the *du-chain* for that definition, the list of all possible uses of that definition. The method is analogous to that discussed in Section 10.5 for computing *ud-chains*, and we leave it to the reader.

The equations for computing *du-chaining* information look exactly like (10.11) with substitution for *def* and *use*. In place of $use[B]$, take the set of *upwards exposed* uses in B , that is, the set of pairs (s, x) such that s is a statement in B which uses variable x and such that no prior definition of x occurs in B . Instead of $def[B]$ take the set of pairs (s, x) such that s is a statement which uses x , s is not in B , and B has a definition of x . These equations are solved by the obvious analog of Algorithm 10.4, and we shall not discuss the matter further.

10.7 CODE-IMPROVING TRANSFORMATIONS

Algorithms for performing the code-improving transformations introduced in Section 10.2 rely on data-flow information. In the last two sections, we have seen how this information can be collected. Here we consider common subexpression elimination, copy propagation, and transformations for moving loop invariant computations out of loops and for eliminating induction variables. For many languages, significant improvements in running time can be achieved by improving code in loops. When such transformations are implemented in a compiler, it is possible to perform some of the transformations together. However, we shall present the ideas underlying the transformations individually.

The emphasis in this section is on global transformations that use information about the program as a whole. As we saw in the last two sections, global data-flow analysis does not usually look at points within basic blocks. Global transformations are therefore not a substitute for local transformations; both must be performed. For example, when we perform global common subexpression elimination we shall only be concerned with whether an expression is generated by a block and not with whether it is recomputed several times within a block.

Elimination of Global Common Subexpressions

The available expressions data-flow problem discussed in the last section allows us to determine if an expression at point p in a flow graph is a common subexpression. The following algorithm formalizes the intuitive ideas presented in Section 10.2 for eliminating common subexpressions.

Algorithm 10.5. Global common subexpression elimination.*Input.* A flow graph with available expression information.*Output.* A revised flow graph.*Method.* For every statement s of the form $x := y + z$ ⁶ such that $y + z$ is available at the beginning of s 's block, and neither y nor z is defined prior to statement s in that block, do the following.

1. To discover the evaluations of $y + z$ that reach s 's block, we follow flow graph edges, searching backward from s 's block. However, we do not go through any block that evaluates $y + z$. The last evaluation of $y + z$ in each block encountered is an evaluation of $y + z$ that reaches s .
2. Create a new variable u .
3. Replace each statement $w := y + z$ found in (1) by

$$\begin{array}{l} u := y + z \\ w := u \end{array}$$

4. Replace statement s by $x := u$. □

Some remarks about this algorithm are in order.

1. The search in step (1) of the algorithm for the evaluations of $y + z$ that reach statement s can also be formulated as a data-flow analysis problem. However, it does not make sense to solve it for all expressions $y + z$ and all statements or blocks, because too much irrelevant information is gathered. Rather we should perform a graph search on the flow graph for each relevant statement and expression.
2. Not all changes made by Algorithm 10.5 are improvements. We might wish to limit the number of different evaluations reaching s found in step (1), probably to one. However, copy propagation, to be discussed next, often allows benefit to be obtained even when several evaluations of $y + z$ reach s .
3. Algorithm 10.5 will miss the fact that $a * z$ and $c * z$ must have the same value in

$$\begin{array}{ll} a := x + y & \text{vs.} \quad c := x + y \\ b := a * z & d := c * z \end{array}$$

because this simple approach to common subexpressions considers only the literal expressions themselves, rather than the values computed by expressions. Kildall [1973] presents a method for catching such equivalences on one pass; we shall discuss the ideas in Section 10.11. However, they can be caught with multiple passes of Algorithm 10.5, and

⁶ Recall we continue to use $+$ as a generic operator.

one might consider repeating it until no further changes occur. If a and c are temporary variables that are not used outside the block in which they appear, then the common subexpression $(x+y)*z$ can be caught by treating the temporary variables specially, as in the next example.

Example 10.18. Suppose that there are no assignments to the array a in the flow graph of Fig. 10.34(a), so we can safely say that $a[t_2]$ and $a[t_6]$ are common subexpressions. The problem is to eliminate this common subexpression.

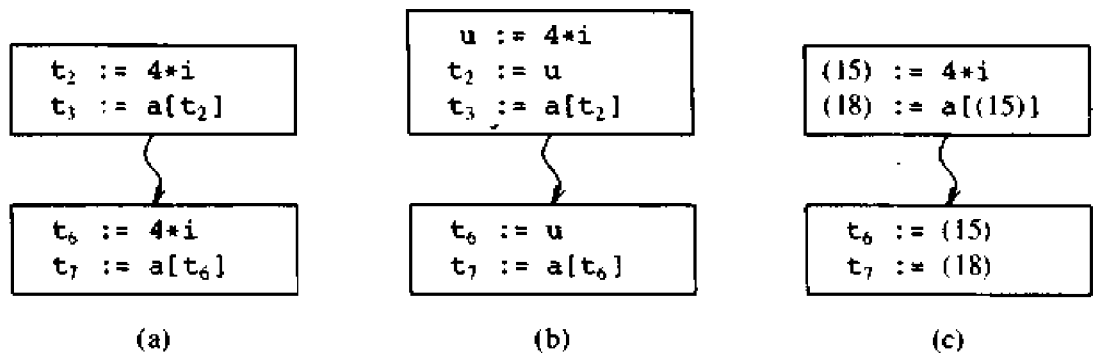


Fig. 10.34. Eliminating the common subexpression $4*i$.

The common subexpression $4*i$ in Fig. 10.34(a) has been eliminated in Fig. 10.34(b). One way to determine that $a[t_2]$ and $a[t_6]$ are also common subexpressions is to replace t_2 and t_6 by u using copy propagation (to be discussed next); both expressions then become $a[u]$, which can be eliminated by reapplying Algorithm 10.5. Note that the same new variable u is inserted in both blocks in Fig. 10.34(b), so local copy propagation is enough to convert both $a[t_2]$ and $a[t_6]$ to $a[u]$.

There is another way, which takes into account the fact that temporary variables are inserted by the compiler and are used only within blocks they appear in. We shall look more closely at the way expressions are represented during the computation of available expressions to get around the fact that different temporary variables may represent the same expression. The recommended technique for representing sets of expressions is to assign a number to each expression and use bit vectors with bit i representing the expression numbered i . The value-numbering techniques of Section 5.2 can be applied during the numbering of expressions to treat temporary variables in a special way.

In more detail, suppose $4*i$ has value number 15. The expressions $a[t_2]$ and $a[t_6]$ will get the same value number if we use the value number 15 rather than the names of the temporary variables t_2 and t_6 . Suppose the resulting value number is 18. Then bit 18 will represent both $a[t_2]$ and $a[t_6]$ during data-flow analysis, and we can determine that $a[t_6]$ is available and can be eliminated. The resulting code is indicated in Fig. 10.34(c). We use (15) and (18) to represent temporaries corresponding to expressions

with those value numbers. Actually, t_6 is useless and would be eliminated during local live variable analysis. Also t_7 , being a temporary, would not be computed itself; rather, uses of t_7 would be replaced by uses of (18). $\square$

Copy Propagation

Algorithm 10.5 just presented, and various other algorithms such as induction-variable elimination discussed later in this section, introduce copy statements of the form $x:=y$. Copies may also be generated directly by the intermediate code generator, although most of these involve temporaries local to one block and can be removed by the dag construction discussed in Section 9.8. It is sometimes possible to eliminate copy statement $s: x:=y$ if we determine all places where this definition of x is used. We may then substitute y for x in all these places, provided the following conditions are met by every such use u of x .

1. Statement s must be the only definition of x reaching u (that is, the ud-chain for use u consists only of s).
2. On every path from s to u , including paths that go through u several times (but do not go through s a second time), there are no assignments to y .

Condition (1) can be checked using ud-chaining information, but what of condition (2)? We shall set up a new data-flow analysis problem in which $in[B]$ is the set of copies $s: x:=y$ such that every path from the initial node to the beginning of B contains the statement s , and subsequent to the last occurrence of s there are no assignments to y . The set $out[B]$ can be defined correspondingly, but with respect to the end of B . We say copy statement $s: x:=y$ is *generated* in block B if s occurs in B and there is no subsequent assignment to y within B . We say $s: x:=y$ is *killed* in B if x or y is assigned there and s is not in B . The notion that assignments to x "kill" $x:=y$ is familiar from reaching definitions, but the idea that assignments to y do so is special to this problem. Note the important consequence of the fact that different assignments $x:=y$ kill each other; $in[B]$ can contain only one copy statement with x on the left.

Let U be the "universal" set of all copy statements in the program. It is important to note that different statements $x:=y$ are different in U . Define $c_gen[B]$ to be the set of all copies generated in block B and $c_kill[B]$ to be the set of copies in U that are killed in B . Then the following equations relate the quantities defined:

$$\begin{aligned}
 out[B] &= c_gen[B] \cup (in[B] - c_kill[B]) \\
 in[B] &= \bigcap_{P \text{ a predecessor of } B} out[P] \quad \text{for } B \text{ not initial} \\
 in[B_1] &= \emptyset \quad \text{where } B_1 \text{ is the initial block}
 \end{aligned}
 \tag{10.12}$$

Equations 10.12 are identical to Equations 10.10, if c_kill is replaced by e_kill and c_gen by e_gen . Thus, 10.12 can be solved by Algorithm 10.3, and we shall not discuss the matter further. We shall, however, give an example that exposes some of the nuances of copy optimization.

Example 10.19. Consider the flow graph of Fig. 10.35. Here, $c_gen[B_1] = \{x:=y\}$ and $c_gen[B_3] = \{x:=z\}$. Also, $c_kill[B_2] = \{x:=y\}$, since y is assigned in B_2 . Finally, $c_kill[B_1] = \{x:=z\}$ since x is assigned in B_1 , and $c_kill[B_3] = \{x:=y\}$ for the same reason.

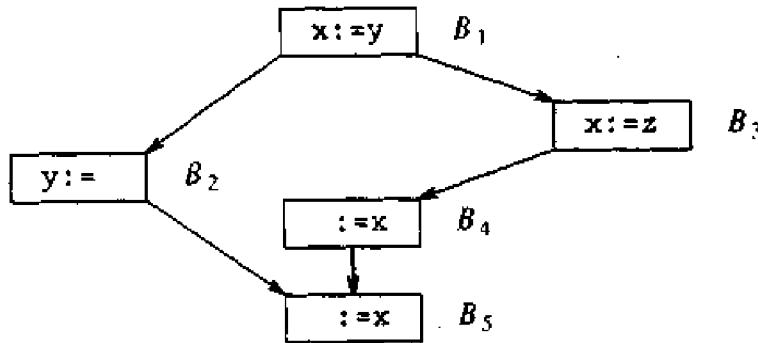


Fig. 10.35. Example flow graph.

The other c_gen 's and c_kill 's are $\emptyset$. Also, $in[B_1] = \emptyset$ by Equations 10.12. Algorithm 10.3 in one pass determines that

$$in[B_2] = in[B_3] = out[B_1] = \{x:=y\}$$

Likewise, $out[B_2] = \emptyset$ and

$$out[B_3] = in[B_4] = out[B_4] = \{x:=z\}$$

Finally, $in[B_5] = out[B_2] \cap out[B_4] = \emptyset$.

We observe that neither copy $x:=y$ nor $x:=z$ "reaches" the use of x in B_5 , in the sense of Algorithm 10.5. It is true but irrelevant that both these definitions of x "reach" B_5 in the sense of reaching definitions. Thus, neither copy may be propagated, as it is not possible to substitute y (respectively z) for x in all uses of x that definition $x:=y$ (respectively $x:=z$) reaches. We could substitute z for x in B_4 , but that would not improve the code. $\square$

We now specify the details of the algorithm to remove copy statements.

Algorithm 10.6. Copy propagation.

Input. A flow graph G , with ud-chains giving the definitions reaching block B , and with $c_in[B]$ representing the solution to Equations 10.12, that is, the set of copies $x:=y$ that reach block B along every path, with no assignment to x or y following the last occurrence of $x:=y$ on the path. We also need du-

chains giving the uses of each definition.

Output. A revised flow graph.

Method. For each copy $s: x := y$ do the following.

1. Determine those uses of x that are reached by this definition of x , namely, $s: x := y$.
2. Determine whether for every use of x found in (1), s is in $c_in[B]$, where B is the block of this particular use, and moreover, no definitions of x or y occur prior to this use of x within B . Recall that if s is in $c_in[B]$, then s is the only definition of x that reaches B .
3. If s meets the conditions of (2), then remove s and replace all uses of x found in (1) by y . $\square$

Detection of Loop-Invariant Computations

We shall make use of ud-chains to detect those computations in a loop that are *loop-invariant*, that is, whose value does not change as long as control stays within the loop. As discussed in Section 10.4, a loop is a region consisting of a set of blocks with a header that dominates all the other blocks, so the only way to enter the loop is through the header. We also require that a loop have at least one way to get back to the header from any block in the loop.

If an assignment $x := y + z$ is at a position in the loop where all possible definitions of y and z are outside the loop (including the special case where y and/or z is a constant), then $y + z$ is loop-invariant because its value will be the same each time $x := y + z$ is encountered, as long as control stays within the loop. All such assignments can be detected from the ud-chains, that is, a list of all definition points of y and z reaching assignment $x := y + z$.

Having recognized that the value of x computed at $x := y + z$ does not change within the loop, suppose there is another statement $v := x + w$, where w could only have been defined outside the loop. Then $x + w$ is also loop-invariant.

We can use the above ideas to make repeated passes over the loop, discovering more and more computations whose value is loop-invariant. If we have both ud- and du-chains, we do not even have to make repeated passes over the code. The du-chain for definition $x := y + z$ will tell us where this value of x could be used, and we need only check among these uses of x , within the loop, that use no other definition of x . These loop-invariant assignments may be moved to the preheader, provided their operands besides x are also loop-invariant, as discussed in the next algorithm.

Algorithm 10.7. Detection of loop-invariant computations.

Input. A loop L consisting of a set of basic blocks, each block containing a sequence of three-address statements. We assume ud-chains, as computed in Section 10.5, are available for the individual statements.

Output. The set of three-address statements that compute the same value each time executed, from the time control enters the loop L until control next leaves L .

Method. We shall give a rather informal specification of the algorithm, trusting that the principles will be clear.

1. Mark "invariant" those statements whose operands are all either constant or have all their reaching definitions outside L .
2. Repeat step (3) until at some repetition no new statements are marked "invariant."
3. Mark "invariant" all those statements not previously so marked all of whose operands either are constant, have all their reaching definitions outside L , or have exactly one reaching definition, and that definition is a statement in L marked invariant. $\square$

Performing Code Motion

Having found the invariant statements within a loop, we can apply to some of them an optimization known as *code motion*, in which the statements are moved to the preheader of the loop. The following three conditions ensure that code motion does not change what the program computes. None of the conditions is absolutely essential; we have selected these conditions because they are easy to check and apply to situations that occur in real programs. We shall later discuss the possibility of relaxing the conditions.

The conditions for statement $s: x := y + z$ are:

1. The block containing s dominates all *exit* nodes of the loop, where an exit of a loop is a node with a successor not in the loop.
2. There is no other statement in the loop that assigns to x . Again, if x is a temporary assigned only once, this condition is surely satisfied and need not be checked.
3. No use of x in the loop is reached by any definition of x other than s . This condition too will be satisfied, normally, if x is a temporary.

The next three examples motivate the above conditions.

Example 10.20. Moving a statement that need not be executed within a loop to a position outside the loop can change what the program computes, as we show using Fig. 10.36. This observation motivates condition (1), since a statement that dominates all the exits cannot fail to be executed, assuming the loop does not run forever.

Consider the flow graph shown in Fig. 10.36(a). B_2 , B_3 , and B_4 form a loop with header B_2 . Statement $i := 2$ in B_3 is clearly loop-invariant. However, B_3 does not dominate B_4 , the only loop exit. If we move $i := 2$ to a newly created preheader B_6 , as shown in Fig. 10.36(b), we may change the

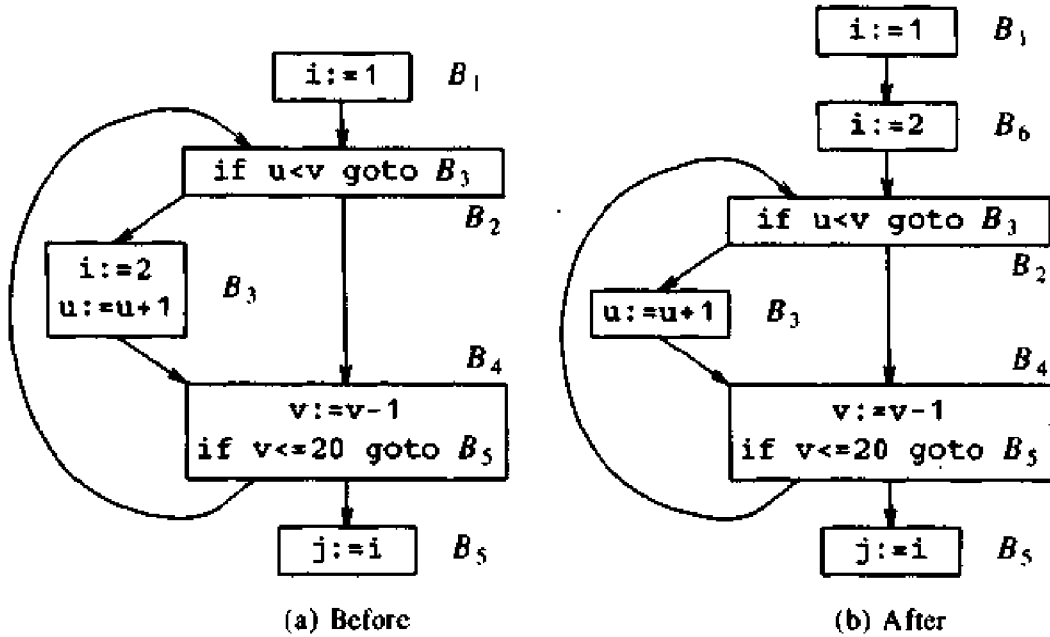


Fig. 10.36. Example of illegal code motion.

value assigned to j in B_5 , in those cases where B_3 never gets executed. For example, if $u=30$ and $v=25$ when B_2 is first entered, Fig. 10.36(a) sets j to 1 at B_5 , since B_3 is never entered, while Fig. 10.36(b) sets j to 2. $\square$

Example 10.21. Condition (2) is required when there is more than one assignment to x in the loop. For example, the flow graph structure in Fig. 10.37 is the same as that in Fig. 10.36(a), and we have the option of creating a preheader B_6 as in Fig. 10.36(b).

Since B_2 in Fig. 10.37 dominates the exit B_4 , condition (1) does not prevent $i:=3$ from being moved to the preheader B_6 . However, if we do so, we shall set i to 2 whenever B_3 is executed, and i will have value 2 when we reach B_5 , even if we follow a sequence such as $B_2 \rightarrow B_3 \rightarrow B_4 \rightarrow B_2 \rightarrow B_4 \rightarrow B_5$. For example, consider what happens if v is 22 and u is 21 when B_2 is first reached. If $i:=3$ is in B_2 , we set j to 3 at B_5 , but if $i:=3$ is removed to the preheader, we set j to 2. $\square$

Example 10.22. Now let us consider rule (3). The use $k:=i$ in block B_4 of Fig. 10.38 is reached by $i:=1$ in block B_1 , as well as by $i:=2$ in B_3 . Thus, we could not move $i:=2$ to the preheader, because the value of k reaching B_5 would change in the case $u \geq v$. For example, if $u=v=0$ then k is set to 1 in the flow graph of Fig. 10.38, but if $i:=2$ is moved to the preheader, we set k to 2 once and for all. $\square$

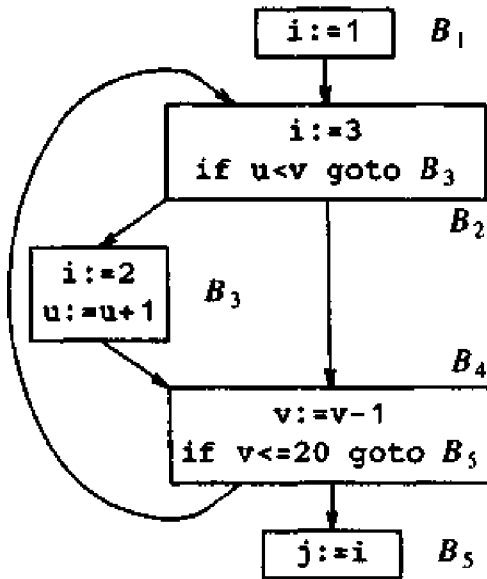


Fig. 10.37. Illustrating condition (2).

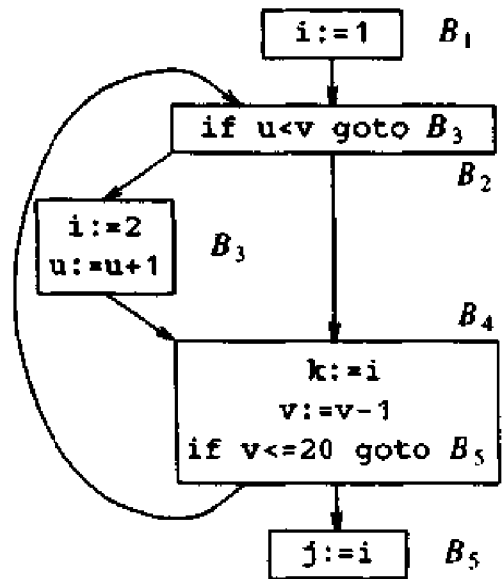


Fig. 10.38. Illustrating condition (3).

Algorithm 10.8. Code motion.

Input. A loop L with ud-chaining information and dominator information.

Output. A revised version of the loop with a preheader and (possibly) some statements moved to the preheader.

Method.

1. Use Algorithm 10.7 to find loop-invariant statements.
2. For each statement s defining x found in step (1), check:
 - i) that it is in a block that dominates all exits of L ,
 - ii) that x is not defined elsewhere in L , and
 - iii) that all uses in L of x can only be reached by the definition of x in statement s .
3. Move, in the order found by Algorithm 10.7, each statement s found in (1) and meeting conditions (2i), (2ii), and (2iii), to a newly created preheader, provided any operands of s that are defined in loop L (in the case that s was found in step (3) of Algorithm 10.7) have previously had their definition statements moved to the preheader. $\square$

To understand why no change to what the program computes can occur, conditions (2i) and (2ii) of Algorithm 10.8 assure that the value of x computed at s must be the value of x after any exit block of L . When we move s to the preheader, s will still be the definition of x that reaches the end of any exit block of L . Condition (2iii) assures that any uses of x within L did, and will continue to, use the value of x computed by s .

To see why the transformation cannot increase the running time of the program, we have only to note that condition (2i) ensures that s is executed at least once each time control enters L . After code motion, it will be executed exactly once in the preheader and not at all in L whenever control enters L .

Alternative Code Motion Strategies

We can relax condition (1) somewhat if we are willing to take the risk that we may actually increase the running time of the program a bit; of course, we never change what the program computes. The relaxed version of code-motion condition (1) [i.e., item 2(i) in Algorithm 10.8] is that we may move a statement s assigning x only if:

- 1'. The block containing s either dominates all exits of the loop, or x is not used outside the loop. For example, if x is a temporary variable, we can be sure (in many compilers) that the value will be used only in its own block. In general, live variable analysis is needed to tell if x is live at any exit from the loop.

If Algorithm 10.8 is modified to use condition (1'), occasionally the running time will increase slightly, but we can expect to do reasonably well on the average. The modified algorithm may move to the preheader certain computations that might not be executed in the loop. Not only does this risk slowing down the program significantly, it may also cause an error in certain circumstances. For example, the evaluation of a division x/y in a loop may be preceded by a test to see whether $y=0$. If we move x/y to the preheader, a division by zero may occur. For this reason, it is unwise to use condition (1') unless optimization may be inhibited by the programmer, or we apply the stricter condition (1) to division statements.

Even if none of conditions (2i), (2ii), and (2iii) of Algorithm 10.8 are met by an assignment $x:=y+z$, we can still take the computation $y+z$ outside a loop. Create a new temporary t , and set $t:=y+z$ in the preheader. Then replace $x:=y+z$ by $x:=t$ in the loop. In many cases we can then propagate out the copy statement $x:=t$, as discussed earlier in this section. Note that if condition (2iii) of Algorithm 10.8 is satisfied, that is, if all uses of x in loop L are defined at $x:=y+z$ (now $x:=t$), then we can surely remove statement $x:=t$ by replacing uses of x in L by uses of t and placing $x:=t$ after each exit of the loop.

Maintaining Data-Flow Information After Code Motion

The transformations of Algorithm 10.8 do not change ud-chaining information, since by conditions (2i), (2ii), and (2iii), all uses of the variable assigned by a moved statement s that were reached by s are still reached by s from its new position. Definitions of variables used by s are either outside L , in which case they reach the preheader, or they are inside L , in which case by step (3) they were moved to the preheader ahead of s .